

SMART Car Supreme

Senior Project 2025-2026



Team Members

Brayden Kimura,
Joshua Oliveira,
Amelia Wirick,
Johnathan Yi

Project Advisor

Hossein Asghari, Ph.D

Department of Electrical and Computer Engineering
Seaver College of Science and Engineering Loyola
Marymount University
Los Angeles, CA

Abstract

The SMART Car Supreme is a 1/10-scale autonomous vehicle platform developed to demonstrate real-time lane-following using an integrated perception, control, and embedded systems architecture. This project builds upon previous iterations of Loyola Marymount University's SMART Car by improving system reliability, perception accuracy, and hardware modularity.

The system utilizes a forward-facing camera and a convolutional neural network (U-Net) running on an NVIDIA Jetson Orin Nano to detect lane boundaries and generate a centerline representation of the road. A Pure Pursuit controller computes steering commands based on a dynamically selected lookahead point, enabling smooth and stable trajectory tracking. These control commands are transmitted over a CAN bus to an Arduino microcontroller, which converts them into PWM signals for steering and throttle actuation.

To improve system organization and scalability, the electronics were redesigned using custom printed circuit boards (PCBs) for power distribution and communication. This reduces wiring complexity and provides a cleaner, more maintainable hardware layout compared to prior implementations.

The vehicle was successfully demonstrated performing autonomous lane-following on a closed test track, maintaining stable control within the defined performance constraints. Testing validated the integration of perception, control, communication, and actuation subsystems. However, performance was affected by environmental factors such as lighting variability and system latency, particularly at higher speeds.

Overall, the SMART Car Supreme demonstrates a complete real-time autonomous driving pipeline and provides a scalable foundation for future enhancements, including obstacle detection, dynamic speed control, and multi-sensor fusion.



Contents

1	Introduction	4
2	Project Objectives	5
2.1	Background Information	5
2.2	Customer Requirements	7
3	Proposed Solution	8
3.1	Trades Leading to Proposed Solution	8
3.2	Technical Requirements	16
3.3	Systems Description	22
3.4	Standards and Constraints	24
3.5	Design Impact	25
4	Electrical Design	25
4.1	Schematic	25
4.2	Wiring and Cable Diagram	26
4.3	Printed Circuit Board (PCB) Layout	27
4.4	Bill of Materials	28
4.5	Mechanical Images	29
4.6	System Design and Control Workflow	31
4.7	Cost Estimates	31
5	Experimental Test and Demonstration	32
5.1	Tests Used to Benchmark Performance	32
5.2	Working Prototype	32
5.3	Demonstration	33
5.4	Data Analytics	34
5.5	Customer Requirements	34
6	Ethics Consideration	35
7	Contributions to ABET Program, LMU Values, Diversity, Social Community, Multidisciplinary, IEEE Values	35
7.1	ABET Program Contributions	35
7.2	LMU Values	36
7.3	IEEE Values	37
8	Conclusion	37
9	Suggestions	38
10	References	38
11	Appendices	39
11.1	Teammate Roles and Responsibilities	39
11.2	Source Code and Supporting Files	39

1 Introduction

Autonomous vehicles (AVs) represent a major advancement in modern engineering, with the potential to significantly improve transportation safety, accessibility, and efficiency. By reducing reliance on human drivers, AV systems aim to minimize risks associated with distracted driving, fatigue, and inconsistent decision-making. In recent years, companies such as Waymo, Tesla, Mercedes-Benz, and General Motors have made substantial progress by developing advanced driver-assistance and partial automation systems. While these solutions demonstrate strong performance in controlled environments, they remain limited by high computational demands, sensitivity to environmental conditions, and reliance on predefined operational domains.

The Society of Automotive Engineers (SAE) defines six levels of driving automation, ranging from Level 0 (fully manual control) to Level 5 (full autonomy in all conditions). Most commercially available systems operate between Levels 2 and 4, where vehicles can assist with steering, acceleration, and braking but still require human oversight or are restricted to specific scenarios. This gap between current capabilities and full autonomy highlights ongoing challenges in perception, decision-making, and system integration.

Academic platforms provide a controlled and cost-effective environment to explore these challenges at a smaller scale. The SMART Car project at Loyola Marymount University uses a 1/10-scale RC vehicle as a testbed for investigating lane detection, control systems, and autonomous navigation. Previous iterations of the project demonstrated basic path-following and semi-autonomous behavior; however, they were limited by complex wiring, poor modularity, and difficulty in debugging and expanding the system.

The SMART Car Supreme project builds upon this foundation by redesigning both the hardware and software architecture to improve reliability, modularity, and performance. The primary objective of this iteration is to enable stable and reliable lane-following around Pereira Quad using a streamlined perception–control–actuation pipeline. In addition to improving autonomy, the system emphasizes clean hardware integration, CAN-based communication, and scalable design practices to support future enhancements such as obstacle detection and multi-sensor integration.

2 Project Objectives

2.1 Background Information

Autonomous driving technologies have the potential to significantly improve quality of life by enhancing mobility, increasing road safety, and reducing transportation barriers for individuals with disabilities or age-related limitations. By shifting responsibility from human drivers to automated systems, autonomous vehicles (AVs) aim to reduce risks associated with distraction, impaired driving, and inconsistent human decision-making. These benefits have driven substantial commercial and academic investment in developing more capable and reliable autonomous systems.

Despite rapid progress, fully autonomous driving has not yet been achieved. Companies such as Waymo, Tesla, Mercedes-Benz, and General Motors have introduced advanced driver-assistance and partial automation systems; however, these solutions remain constrained to specific operational design domains or require human supervision. The Society of Automotive Engineers (SAE) defines six levels of driving automation, ranging from Level 0 (no automation) to Level 5 (full autonomy under all conditions). This framework is widely used to describe both current capabilities and remaining challenges in autonomous system development.

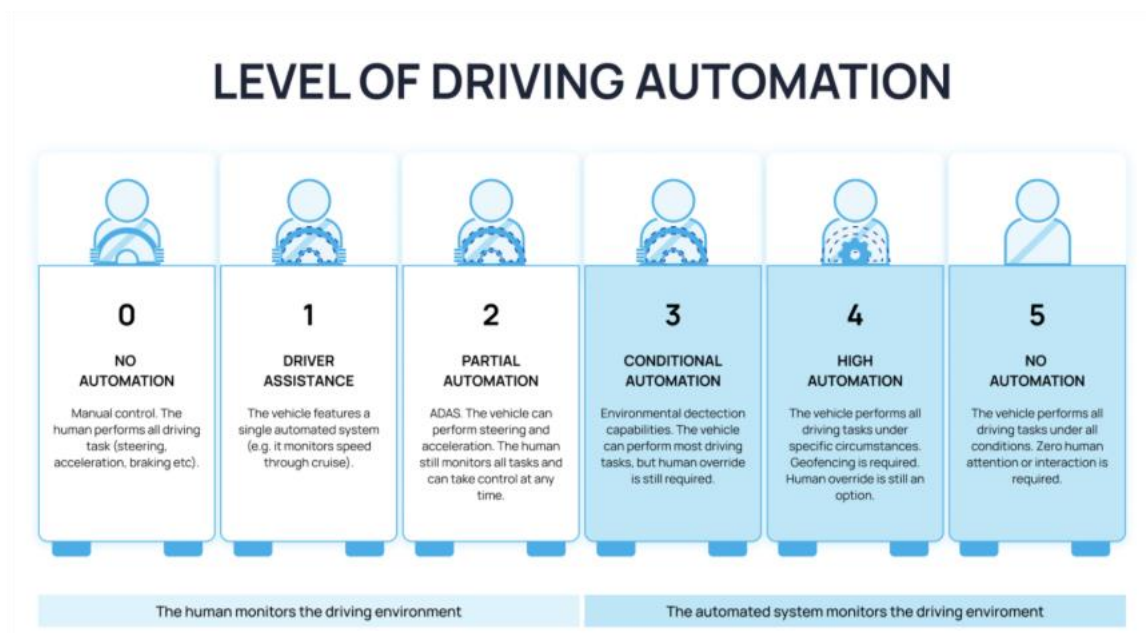


Figure 1: Levels of driving automation [2]

To further contextualize the capabilities and limitations of current systems, Table 1 summarizes key characteristics of leading commercial autonomous platforms, including their level of automation and primary strengths and constraints.

Table 1: Autonomous Driving Solutions

Solution	Level of Automation	Pros	Cons
Waymo Driver	4	-does not need a driver in the driver's seat -anti-collision technology -comes up with its own path -automatically brakes, steers, accelerates, and changes lanes [3]	-relies on geofencing -allows for driver override [3]
Tesla Full Self Driving	2	-automatically brakes, steers, accelerates, and changes lanes -comes up with its own path -anti-collision technology [4]	-requires constant driver supervision -does not completely automatically prevent collision [4]
Mercedes-Benz Drive Pilot	3	-automatically brakes, steers, and accelerates -driver can take hands off the wheel and divert attention from the road -anti-collision technology [5]	-requires daylight and clear weather -under 40 mph speeds on freeway -requires driver in the seat -no construction zone present [5]
General Motors Super Cruise	2	-automatically brakes, steers, accelerates, and changes lanes -anti-collision technology [6]	-does not follow a set path -requires an attentive driver [6]

Although these systems demonstrate strong performance in controlled environments, they highlight persistent challenges in perception of reliability, environmental generalization, and system safety. These limitations emphasize the need for continued research in sensing, control algorithms, and system integration.

Academic platforms provide a controlled and scalable environment to explore these challenges. Loyola Marymount University's SMART Car project uses a 1/10-scale RC vehicle as a testbed for autonomous navigation around Pereira Quad. Previous iterations demonstrated basic path-following, camera-based lane detection, and obstacle response. However, several limitations have persisted:

- Complex and disorganized wiring, increasing failure rates and complicating debugging
- Lack of modularity, making system upgrades and maintenance difficult
- Limited sensing accuracy, particularly for lane detection and obstacle avoidance
- Inconsistent performance after extended use or hardware modifications

These issues define the central problem addressed in this project:

The existing autonomous RC car must be redesigned to improve reliability, sensing capability, modularity, and overall autonomous navigation performance while supporting modern AV system design practices.

The SMART Car Supreme project builds upon this foundation by developing a more robust and modular system capable of reliable lane-following, improved sensing, and cleaner hardware integration.

2.2 Customer Requirements

The SMART Car Supreme project represents the fifth iteration of Loyola Marymount University's autonomous RC vehicle platform. While previous versions demonstrated partial autonomy, including basic path-following and obstacle detection, system reliability degraded over time due to hardware limitations and poor system organization. As a result, the current customer—represented by the LMU Electrical and Computer Engineering Department—requires a more reliable, modular, and scalable platform capable of supporting modern autonomous driving techniques.

Based on analysis of prior system limitations, input from faculty advisors, and benchmarking against commercial autonomous systems, the customer requirements were defined around four key pillars: safety, navigation accuracy, system modularity, and demonstration readiness.

Customer Requirements (Qualitative)

1. Autonomous Navigation
The system shall autonomously complete a predefined loop around Pereira Quad without human intervention.
2. Lane Following
The vehicle shall remain centered within the lane and follow the marked path with minimal deviation.
3. Modular System Architecture
The electronics shall be redesigned using printed circuit boards (PCBs) and organized wiring to improve reliability and maintainability.

4. Clean Hardware Design
All wiring, sensors, PCBs, and mounting structures shall be physically organized, accessible, and robust for outdoor demonstration.
5. Safe Operation
The vehicle's speed, braking, and steering shall ensure predictable and safe behavior during public demonstrations.

Customer Requirements (Quantitative Performance Metrics)

To enable objective evaluation, qualitative requirements were translated into measurable performance metrics.

Table 2: Customer Requirements

Customer Requirement	Performance Metric
Lane Following	Car remains within 6 inches of lane boundary
Autonomous Navigation	Complete ~350 ft Pereira Quad loop
Safe Operation	Maximum speed of 3 ft/sec
Modular Hardware	PCBs $\leq 4 \times 5$ in with organized wiring

These performance metrics define the minimum acceptable criteria for system validation and provide clear benchmarks for evaluating autonomous performance.

Connection to Project Milestones

The customer requirements directly informed the project milestones used to guide development:

Milestone 1: Validate RC car platform functionality

Milestone 2: Replace outdated sensors and reorganize wiring

Milestone 3: Transition electronics to modular PCB design

Milestone 4: Achieve autonomous navigation around Pereira Quad

Milestone 5: Implement braking and safety behavior for obstacle avoidance

These milestones establish a structured development path and ensure alignment between system requirements and implementation goals.

3 Proposed Solution

3.1 Trades Leading to Proposed Solution

The design of the SMART Car Supreme required evaluation of multiple sensing, computation, and system integration approaches. To meet the project goals of reliability, modularity, and improved perception performance, structured trade studies were conducted across four key subsystems: lane detection, obstacle detection, compute platform selection, and electronics integration.

These trade studies used engineering design methodologies including concept generation, pairwise comparison, Analytic Hierarchy Process (AHP), and weighted decision matrices. This systematic approach ensured that the final architecture was selected based on quantitative performance metrics and alignment with customer requirements.

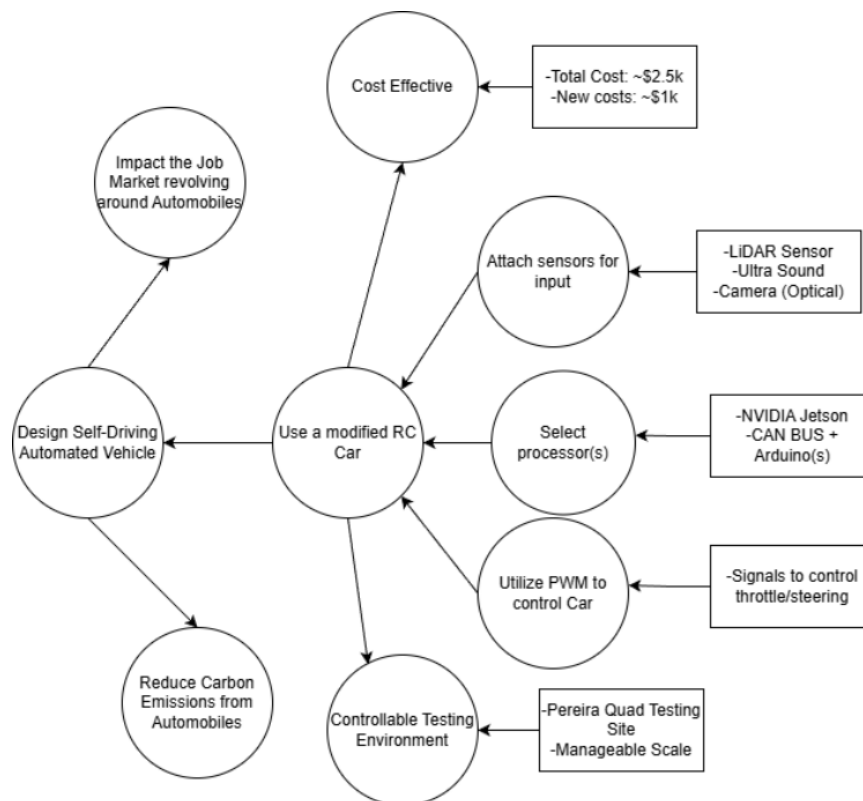
3.1.1 Concept Generation and High-Level Alternatives

Based on customer requirements, the system was decomposed into four primary subsystems:

- Lane detection sensing
- Obstacle detection sensing
- Compute platform
- Electronics integration

To explore possible design solutions, a concept fan was used to expand the design space and identify multiple viable system architectures before narrowing to a final configuration.

Figure 2: Concept Fan for System Decomposition



3.1.2 Lane Detection: Camera vs. LiDAR Trade Study

Table 3 shows the advantages and disadvantages of each possible consideration for lane detection.

Table 3: Camera vs LiDAR comparison

Technology	Advantages	Disadvantages
RGB camera	Low cost, ML-compatible, high resolution	Sensitive to glare and lighting
Depth camera	3D lane geometry	Expensive, less robust outdoors
LiDAR for lane detection	Excellent accuracy	Unnecessary for lane lines; high compute load
Hybrid LiDAR + camera	Strong perception	Too complex for RC-scale platform

Decision:

A camera-based neural lane detection system was selected as the primary approach. This decision is supported by several factors:

- Modern autonomous systems, such as Tesla’s Full Self Driving, rely primarily on vision-based perception
- The Jetson platform provides sufficient compute capability for real-time neural inference
- The lane environment at Pereira Quad is visually structured and consistent
- LiDAR provides unnecessary complexity for detecting lane markings

As a result, LiDAR is reserved for obstacle detection, while lane detection is performed using a vision-based approach.

Table 4: Weight Calculations for Camera

Criteria	Saturation Detection	Glare Prevention	Size	Build Quality	Video Quality	Availability	Cost	Geometric Mean	Weights
Saturation Detection	1	1	3	3	1	6	4	2.15523	0.24390
Glare Prevention	1	1	3	3	2	7	5	2.51134	0.28421
Size	1/3	1/3	1	1/2	1/3	3	1	0.66172	0.07489
Build Quality	1/3	1/3	2	1	1/3	3	1	0.80665	0.09129
Video Quality	1	1/2	3	3	1	4	3	1.76801	0.20008
Availability	1/6	1/7	1/3	1/3	1/4	1	1/2	0.31825	0.03602
Cost	1/4	1/5	1	1	1/3	2	1	0.61515	0.06962

Table 5: Decision Matrix for which camera

		IMX 477	4K HDR GMSL2
Saturation Detection	0.24	0.4	0.6
Glare Prevention	0.28	0.35	0.65
Size	0.07	0.65	0.35
Build Quality	0.09	0.45	0.55
Video Quality	0.20	0.35	0.65
Availability	0.04	0.85	0.15
Cost	0.07	0.75	0.25
Score		0.439644	0.560355508

Table 5 shows the matrix comparing two camera options—IMX477 and a 4K HDR GMSL2 automotive camera—based on properties essential for lane detection and visual perception.

The criteria include glare prevention, saturation handling, video quality, build quality, physical size, and cost. Because lane detection depends heavily on consistent visual clarity, glare prevention and saturation detection were assigned the highest weights through the AHP process.

The results show that the 4K HDR GMSL2 camera achieved the highest overall score (0.5603), particularly excelling in glare handling, saturation robustness, and video quality—key factors for reliable lane detection in outdoor environments.

The IMX477 camera scored lower due to reduced performance in high dynamic lighting conditions, despite its lower cost and compact size. Based on these results, the GMSL2 camera is the optimal choice for lane detection. However, due to time and budget constraints, the IMX477 camera was selected for implementation, with the GMSL2 identified as a future upgrade.

3.1.3 Obstacle Detection: Ultrasound vs. LiDAR vs. Vision Fusion

Multiple sensing approaches were evaluated for obstacle detection, including ultrasonic sensors, LiDAR, vision-based detection, and multi-sensor fusion. Each option was assessed based on accuracy, range, robustness, and integration complexity.

Table 6 covers alternatives considered included ultrasonic sensors, short-range LiDAR, vision-only detection, and multi-sensor fusion.

Table 6: Ultrasound vs. LiDAR vs. Vision Fusion comparisons

Option	Advantages	Disadvantages
Ultrasonic sensors	Low cost, simple	Poor range, unreliable outdoors
LiDAR	High accuracy, long range, robust	Higher cost, higher processing needs
Camera-only detection	Flexible, multi-purpose	Requires dataset, lighting-dependent

Decision:

A hybrid architecture was selected consisting of:

- Jetson Orin Nano for perception, LiDAR processing, and decision-making
- Arduino CAN-based nodes for deterministic PWM control

This architecture was chosen because:

- Jetson platforms provide the GPU acceleration required for real-time perception
- Arduino nodes offer stable, low-latency actuation
- CAN communication provides a robust alternative to traditional wiring
- The design aligns with industry architectures that separate high-level computation from low-level control

Table 7: Weight Calculations for LiDAR

Criteria	Dimensions	Accuracy	Size	Build Quality	Availability	Hz	Range	Cost	Geometric Mean	Weights
Dimensions	1	2	5	4	7	3	1/2	4	2.53025	0.24082
Accuracy	1/2	1	4	3	6	2	1/3	3	1.56508	0.14896
Size	1/5	1/4	1	1/2	3	1/2	1/5	1	0.54247	0.05163
Build Quality	1/4	1/3	2	1	3	1/2	1/5	3	0.74989	0.07137
Availability	1/7	1/6	1/3	1/3	1	1/6	1/9	1/3	0.26017	0.02476
Hz	1/3	1/2	2	2	6	1	1	3	1.56508	0.14896
Range	2	3	5	5	9	1	1	4	2.73708	0.26050
Cost	1/4	1/3	1	1/3	3	1/3	1/4	1	0.55695	0.05301

The LiDAR decision matrix evaluates four candidate LiDAR units—SLAMTEC C1, Unitree 4D LiDAR L2, CRATUS JT16, and CRATUS JT128—using criteria that directly impact autonomous perception performance. These criteria include accuracy, range, scan frequency (Hz), build quality, physical dimensions, and cost. Each criterion was weighted using the Analytic Hierarchy Process (AHP), shown earlier in the pairwise comparison table, to reflect its relative importance in real driving scenarios.

For example, range and accuracy received the highest weights because our vehicle must reliably detect obstacles and map its surroundings at varying distances. Cost and size were weighted lower because performance is more important for safety than minimal size or price.

By multiplying each device's ratings by the criterion weights, we generated overall scores for each LiDAR. The CRATUS JT128 scored the highest overall (0.3185), indicating that it provides the best balance of long-range performance, robustness, and scanning capability. The Unitree 4D LiDAR L2 also performed well, especially in availability and cost, but did not match the JT128 in core sensing performance. Therefore, the decision matrix supports selecting the CRATUS JT128 as the LiDAR best suited for our autonomous RC vehicle.

3.1.4 Compute Platform Trade Study: Jetson vs. Alternatives

Selecting an appropriate compute platform was critical for supporting real-time perception, control, and sensor integration. The system requires sufficient computational capability to process camera data, perform neural network inference, interface with LiDAR sensors, and execute control algorithms with low latency.

To evaluate potential compute solutions, several embedded platforms were compared, including the Arduino Uno, Raspberry Pi 5, Jetson Nano, and Jetson Orin Nano. The comparison considered criteria including SDK and driver support, available I/O interfaces, compute performance, real-time responsiveness, power consumption, memory capacity, availability, cost, and physical form factor.

SDK support and compute performance received the highest weighting because autonomous driving applications depend heavily on GPU-accelerated perception pipelines, neural inference, and reliable sensor integration. I/O capability was also heavily weighted due to the need for high-speed interfaces such as USB 3.0, CAN bus, and serial communication for cameras, LiDAR, and control hardware.

Table 8: Weight calculation for Jetson

Criteria	SDK/Driver Support	I/O & Buses	Compute Power	Real-Time/Latency	Power & Thermals	Memory	Cost	Availability	Form Factor	Geometric Mean	Weights
SDK/Driver Support	1	2	1/2	2	3	3	5	5	4	2.29983	0.20378
I/O & Buses	1/2	1	1/2	2	3	2	4	4	3	1.73707	0.15391
Compute Power	2	2	1	3	3	3	5	5	4	2.80645	0.24866
Real-Time/Latency	1/2	1/2	1/3	1	2	2	3	3	3	1.27651	0.11311
Power & Thermals	1/3	1/3	1/3	1/2	1	2	3	3	3	1	0.08860
Memory	1/3	1/2	1/3	1/2	1/2	1	3	3	2	0.85724	0.07596
Cost	1/5	1/4	1/5	1/3	1/3	1/3	1	3	2	0.50722	0.04494
Availability	1/5	1/4	1/5	1/3	1/3	1/3	1/3	1	2	0.39734	0.03521
Form Factor	1/4	1/3	1/4	1/3	1/3	1/2	1/2	1/2	1	0.40441	0.03583

The weighted decision matrix shown in Table 9 compares each candidate platform against the selected criteria.

Table 9: Chip Decision Matrix

		Arduino Uno	Raspberry Pi 5	Jetson Nano	Jetson Orin
SDK/Driver Support	0.20				
I/O & Buses	0.15	0.15	0.2	0.3	0.35
Compute Power	0.25	0.42	0.18	0.21	0.19
Real-Time/Latency	0.11	0.14	0.28	0.28	0.3
Power & Thermals	0.09	0.28	0.28	0.28	0.16

Memory	0.08	0.25	0.25	0.25	0.25
Cost	0.04	0.08	0.18	0.44	0.3
Availability	0.04				
Form Factor	0.04	0.85	0.1	0.05	0
Score		0.164089	0.251758089	0.273079	0.311074

The results indicate that Jetson-based platforms provide the strongest overall performance for autonomous perception tasks due to their GPU acceleration, mature SDK ecosystem, and high-bandwidth sensor support. Among the evaluated platforms, the Jetson Orin Nano achieved the highest overall score because it provides the best balance of compute capability, latency performance, and sensor integration support.

Although platforms such as the Raspberry Pi 5 offer lower cost and acceptable general-purpose performance, they lack the GPU acceleration required for efficient neural network inference and advanced perception workloads. Similarly, microcontrollers such as the Arduino Uno are well suited for deterministic low-level control tasks but do not provide sufficient processing capability for real-time perception or sensor fusion.

Decision

Based on the trade study results, a hybrid architecture was selected consisting of:

- Jetson Orin Nano for perception, neural inference, LiDAR processing, and high-level decision-making
- Arduino CAN-based nodes for deterministic PWM steering and throttle control

This architecture was selected for several reasons:

- Jetson platforms provide the GPU acceleration required for real-time lane detection and perception processing
- Arduino nodes provide stable and low-latency hardware control for steering and motor actuation
- CAN bus communication offers improved robustness and modularity compared to direct point-to-point wiring
- The architecture closely mirrors modern autonomous vehicle systems that separate centralized computation from distributed low-level control modules

This combination provides both the computational performance required for autonomous perception and the reliability necessary for real-time actuation and embedded control.

3.1.5 Electronics Integration: Breadboard vs. PCB

Different approaches for system integration were evaluated, including breadboards, perfboards, and custom PCBs. While breadboards allow rapid prototyping, they are unreliable in dynamic environments. Perfboards provide semi-permanent solutions but remain difficult to maintain and replicate.

Custom PCBs were selected as the final solution due to their superior reliability, clean organization, and modularity. PCBs enable structured power distribution, sensor routing, and communication interfaces, directly addressing the wiring and maintenance issues observed in previous SMART Car iterations.

Table 10 shows the advantages and disadvantages of considerations for electronics integration.

Table 10: Breadboard vs. PCB comparisons

Integration Method	Advantages	Disadvantages
Breadboards	Fast prototyping	Unreliable vibration, messy wiring
Perfboards	Semi-permanent	Hard to replicate, difficult debugging
Custom PCBs	Clean, modular, reliable	Requires design effort

Decision:

The team selected custom PCB modules for:

- Power regulation
- Sensor routing
- CAN communication nodes
- Jetson interfacing

This directly addresses prior Smart Car wiring failures and fulfills customer requirements for modularity and maintainability.

3.1.6 Supplementary Design Evaluations (SCAMPER + Pugh Matrix)

To further explore alternative design solutions beyond initial trade studies, structured ideation and evaluation methods were applied. Specifically, SCAMPER analysis was used to expand the design space, followed by a Pugh concept selection matrix to systematically compare key alternatives.

SCAMPER Analysis

The SCAMPER methodology was used to challenge assumptions and generate alternative system configurations. Each category—Substitute, Combine, Adapt, Modify, Put to other use, Eliminate, and Rearrange—was applied to different aspects of the system design to encourage broader exploration of possible solutions.

Key examples of SCAMPER-based considerations include:

- **Substitute:** Evaluate replacing LiDAR with camera-based perception
- **Combine:** Integrate lane detection and collision avoidance into a unified perception system
- **Adapt:** Modify obstacle detection strategies to enable navigation around objects rather than simple stopping
- **Modify:** Adjust lane marker characteristics or detection thresholds to improve robustness
- **Put to other use:** Repurpose existing sensors for multiple perception tasks
- **Eliminate:** Reduce unnecessary power sources and simplify wiring architecture
- **Rearrange:** Minimize component count to improve system reliability and maintainability

The SCAMPER table documents how we applied structured ideation prompts to explore alternative design choices for the smart car system. Each letter of SCAMPER—Substitute,

Combine, Adapt, Modify, Put to other use, Eliminate, and Rearrange—challenged us to rethink elements of the design.

For instance, we examined whether cameras could substitute for LiDAR, whether collision avoidance and lane detection could be combined into a single neural network, or whether unnecessary power sources could be eliminated to simplify wiring. This process encouraged creativity beyond the first “obvious” solution and helped us confirm which ideas were feasible versus impractical. It broadened our design space before narrowing down our final system architecture.

Pugh Concept Selection

- A Pugh concept selection matrix was used to evaluate obstacle detection approaches relative to a LiDAR-based baseline. The alternatives considered included:
- LiDAR
- Camera-only detection
- Ultrasonic sensing

Each approach was evaluated based on criteria such as accuracy, range, robustness, and integration complexity. The results showed that both camera-only and ultrasonic solutions scored lower than LiDAR when compared against the baseline, primarily due to limitations in reliability and environmental performance.

Based on this analysis, LiDAR was confirmed as the most robust and suitable method for obstacle detection in the SMART Car Supreme system.

Final System Architecture

Based on all trade studies, decision matrices, and concept evaluations, the final system architecture consists of:

- Jetson Orin Nano for perception and decision-making
- Arduino CAN-based nodes for low-level actuation
- LiDAR (JT128 or Unitree L2) for obstacle detection
- Vision-based camera system for lane detection
- Custom PCBs for modular and reliable system integration

This configuration satisfies the defined customer requirements by achieving:

- Lane-following accuracy within ± 6 inches
- Obstacle detection and stopping within 12 inches
- Improved system robustness and safety
- Modular and maintainable hardware design
- Reliable autonomous navigation around Pereira Quad

3.2 Technical Requirements

3.2.1 Engineering Requirements Overview

Technical requirements for the SMART Car Supreme were developed by translating customer requirements into measurable engineering specifications. These requirements define the performance thresholds necessary to validate autonomous navigation, lane following, system safety, and modular hardware integration.

The requirements were established through analysis of prior SMART Car iterations, benchmarking against commercial autonomous vehicle systems, and the trade studies presented in Section 3.1. This process ensured that each engineering requirement remained directly connected to customer needs and system-level performance objectives.

The primary engineering requirements include:

- Maintain lane position within ± 6 inches of the marked path
- Limit maximum speed to 3 ft/s for safe operation
- Complete a minimum autonomous driving distance of 350 feet
- Utilize modular PCB-based electronics for maintainability and reliability

Table 11 below summarizes the mapping between customer and engineering requirements, along with justifications for each.

Table 11: System Requirements

Marketing requirements	Engineering requirements	Justification
1, 2	The vehicle shall stay in a designated lane marked with tape, within 6 inches	The car will use lane assist to follow the path around Pereira Quad
5	The maximum speed will be 3 ft/sec	The car will safely drive around the track
1, 2	The car will have a minimum driving length of 350 feet	This is the approximate distance of a lap around Pereira Quad
3, 4	The car will use multiple small PCBs measuring maximum 4 inches by 5 inches	Modular approach to PCB development is better for an iterative process
Marketing requirements: The car will be able to drive autonomously around Pereira Quad. The car is able to follow the predetermined path. The car will have a modular design. The car will have cleaner wiring. The car will have safety features		

Table 11 summarizes the relationship between customer requirements and the engineering specifications used to evaluate system performance. These specifications establish measurable benchmarks that guide implementation and testing.

3.2.2 Engineering–Marketing Trade-off Matrix

Engineering requirements often interact with customer expectations in ways that create competing priorities. To better understand these relationships, an Engineering–Marketing Trade-off Matrix was developed.

Table 12: Engineering-Marketing Trade-off Matrix

		Max Speed of 3 ft/sec	Minimum driving length of 350 ft	Use pcbs	Distance from lane of 6 inch
		+	+	+	+
Lane Keeping	+	↓↓			↑↑ ↑↑
Autonomously Drive Around Pereira Quad	+	↑↑	↑↑ ↑↑		↑↑ ↑↑
safety	+	↓↓			↑↑
Modular design	+			↑↑ ↑↑	
Cleaner wiring	+			↑↑ ↑↑	

The trade-off matrix illustrates how changes in engineering specifications can positively or negatively affect customer requirements. For example, increasing vehicle speed improves perceived system capability and reduces traversal time, but negatively impacts stopping distance, stability, and lane-following accuracy. Similarly, tighter lane-keeping tolerances improve autonomous navigation performance but place greater demands on perception of accuracy and controller responsiveness.

These trade-offs guided the prioritization of safety, lane stability, and system reliability over aggressive speed targets.

3.2.3 Engineering Trade-off Matrix

In addition to customer interactions, engineering requirements also influence one another. An Engineering Trade-off Matrix was developed to identify these subsystem-level relationships and evaluate how modifying one parameter affects overall system performance.

Table 13: Engineering Trade-off Matrix

		Max speed	Distance from lane	Use of PCBs	Minimum driving length
		+	+	+	+
Max speed	+	x	↓↓		
Distance from lane	+	x	x		
Use of PCBs	+	x	x	x	
Minimum driving length	+	x	x	x	x

The matrix highlights several important relationships within the system design. Increasing

vehicle speed negatively impacts lane-following stability and stopping performance, while stricter lane accuracy requirements increase controller and perception complexity. Similarly, modular PCB integration improves maintainability and wiring organization but introduces additional design effort during implementation.

Understanding these engineering trade-offs was important for balancing performance, safety, and maintainability throughout the design process.

3.2.4 Competitive Benchmarking

To ensure that the SMART Car Supreme remained aligned with modern autonomous vehicle principles, the project specifications were compared against commercial systems including Waymo, Mercedes Drive Pilot, and Tesla Full Self Driving.

Table 14: Competitive Benchmarks

	Waymo	Drive Pilot	SMART Car Supreme
Margin to lane	Adjust based on safety program	Depends on lane size and vehicle drive next to it. But usually it is in the middle (lane length/2)	6 inches
Range	Whole dedicated city which has been mapped out	Highway which has been mapped out by Mercz	350 ft
Use of pcbs	yes	yes	yes
Max speed	Posted speed limit based on data	59 mph (in Germany) 39 mph in the US	3 ft/sec

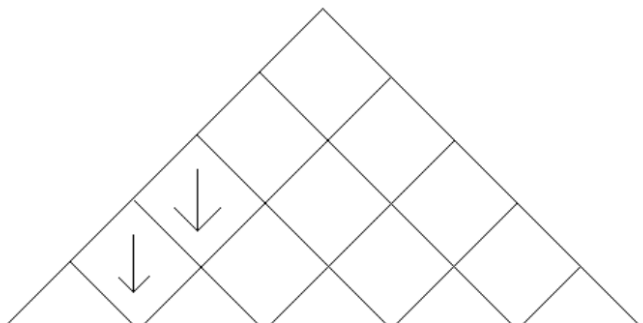
Although commercial autonomous systems operate at significantly larger scales and higher speeds, they rely on many of the same core principles used in the SMART Car Supreme, including lane following, perception-based navigation, and modular electronic architectures.

The SMART Car Supreme adapts these concepts to a 1/10-scale academic platform while maintaining measurable safety and navigation requirements appropriate for the demonstration environment.

3.2.5 Competitive Benchmarking

The House of Quality (HoQ) analysis was used to map customer requirements to engineering specifications and evaluate the relative importance of each subsystem requirement.

Figure 3: House of Quality

						
		Max Speed	Stopping distance	Minimum Distance Covered	Use pcbs	Distance from lane
		+	+	+	+	+
Collision Avoidance	+	⇓	⇑			
Lane Keeping	+	↓				⇑
Autonomously Drive Around Pereira Quad	+	↑	↑	⇑		⇑
safety	+	↓	⇑			↑
Modular design	+				⇑	
Cleaner wiring	+				⇑	
Requirements	X	3 Ft/Sec	1 Ft	350Ft	Modular	6 In

The HoQ analysis identified lane-following accuracy and obstacle avoidance as the most critical system requirements because they directly impact safety and autonomous navigation performance. Cruise speed received moderate weighting to balance system responsiveness with safe operation. Modular electronics and clean wiring also received strong weighting due to the reliability and maintenance issues observed in previous SMART Car iterations.

This analysis guided engineering priorities by emphasizing reliable perception, stable control performance, and maintainable hardware integration.

3.2.6 Requirements Analysis Framework

To ensure that all customer requirements were measurable and verifiable, a requirements analysis framework was developed linking customer needs, engineering specifications, and verification methods.

Table 15 below summarizes the general engineering requirements, showing their relationship to customer needs and verification methods.

Table 15 General Requirements Analysis

Marketing Requirements	Engineering Requirements	Justification / Verification
1, 2	Maintain lane position error $\leq \pm 6$ in	Verified using video tracking and measured lane deviation across test laps
5	Maintain cruise speed ≥ 3 ft/s	Verified by timed 50-ft trials and encoder data
1, 2, 3	Complete one 350-ft autonomous lap	Verified with encoder or GPS logging
3, 4	Modular PCBs $\leq 6 \times 9$ in with CANbus links	Verified by design review and CANbus analyzer diagnostics

Marketing		Requirement	Key:
1	–	Lane	following
2	–	Path	tracking
3	–	Full-loop	completion
4	–	Modular	design
5 – Safety and reliability			

The framework establishes traceability between customer expectations and measurable engineering performance. Each requirement includes a corresponding validation approach, ensuring that system functionality can be objectively tested during final evaluation and demonstration.

3.2.7 Final Engineering Specifications

The final engineering specifications used to evaluate SMART Car Supreme system performance are summarized below.

Table 16 summarizes the final technical specifications that must be met for successful system operation.

Table 16: Final Engineering Specifications

Subsystem	Specification	Target Value	Verification Method
Lane Detection	Lateral deviation from lane	$\leq \pm 6$ in	Video analysis, positional tracking
Speed Control	Cruise velocity	≥ 3 ft/s	Time-distance measurement
Navigation	Complete Pereira Quad loop	~ 350 ft autonomous	GPS/encoder logging
Compute System	Real-time perception latency	< 100 ms	ROS2 profiling + Jetson diagnostics
Communication	CANbus message reliability	$\geq 99\%$ delivery success	CAN analyzer tools
Electronics Integration	PCB dimensions	$\leq 4 \times 5$ in	PCB design review
Power System	Stable 5 V and 12 V rails	$\pm 5\%$ tolerance	Multimeter and load testing

These specifications define the measurable thresholds required for successful system operation and validation. Verification methods including video analysis, timing measurements, CANbus diagnostics, and electrical testing were selected to ensure objective evaluation of each subsystem.

3.2.8 Traceability Summary

Every engineering specification presented in this section is directly traceable to:

- Customer requirements defined in Section 2
- Trade study decisions presented in Section 3.1
- System-level performance goals established during requirements analysis

This traceability ensures that all major design decisions remain connected to measurable project objectives and customer expectations.

3.3 Systems Description

The SMART Car Supreme operates through a real-time perception–control–actuation architecture designed to enable reliable autonomous lane-following behavior. The system integrates camera-based perception, embedded control, CAN bus communication, and low-level actuation into a modular autonomous vehicle platform.

3.3.1 System Architecture

The system uses a centralized compute architecture in which high-level perception and decision-making are performed on the NVIDIA Jetson Orin Nano, while low-level steering and throttle control are handled by distributed Arduino-based CAN nodes.

A forward-facing camera mounted on the vehicle continuously captures image data of the driving environment. This data is processed onboard the Jetson using a convolutional neural network (U-Net architecture) to detect lane boundaries and generate a centerline representing the desired driving path.

The Jetson computes steering and throttle commands based on the detected lane geometry and transmits these commands over the CAN bus to an Arduino microcontroller. The Arduino converts the commands into PWM outputs that actuate the steering servo and drive motor.

This separation between high-level computation and low-level control improves modularity, maintainability, and system reliability while reflecting architectures commonly used in modern autonomous vehicle systems.

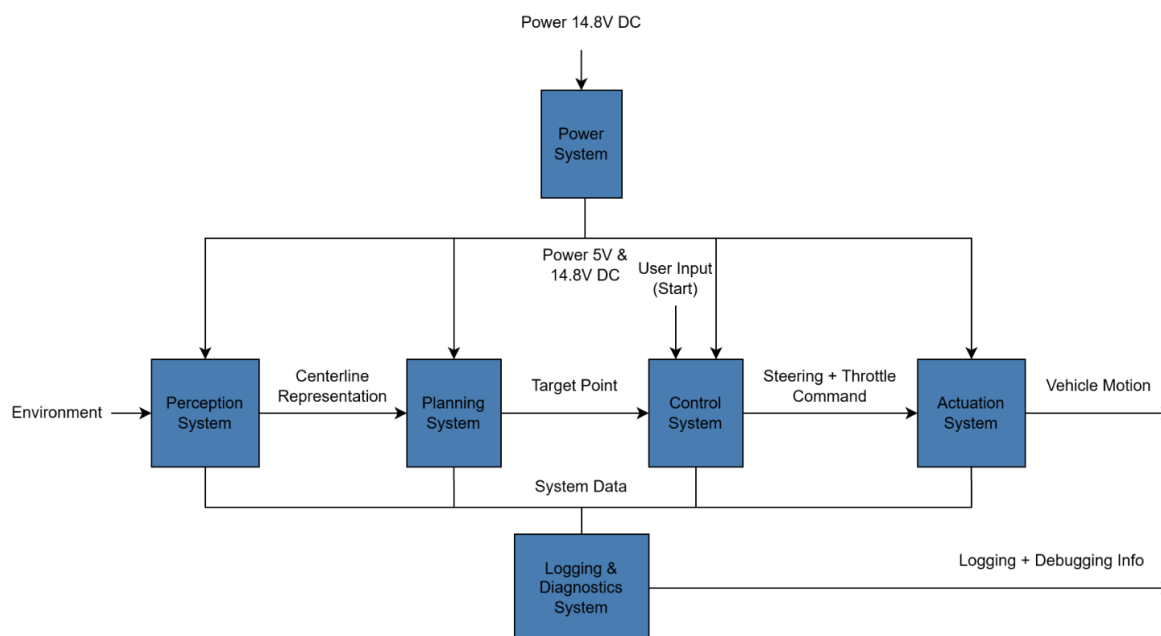


Figure 4: SMART Car Supreme System Architecture

3.3.2 Software Pipeline

The SMART Car Supreme software pipeline follows a perception–planning–control workflow.

The perception stage begins with camera image acquisition and lane detection using the U-

Net neural network. The resulting lane boundaries are converted into a centerline representation of the roadway.

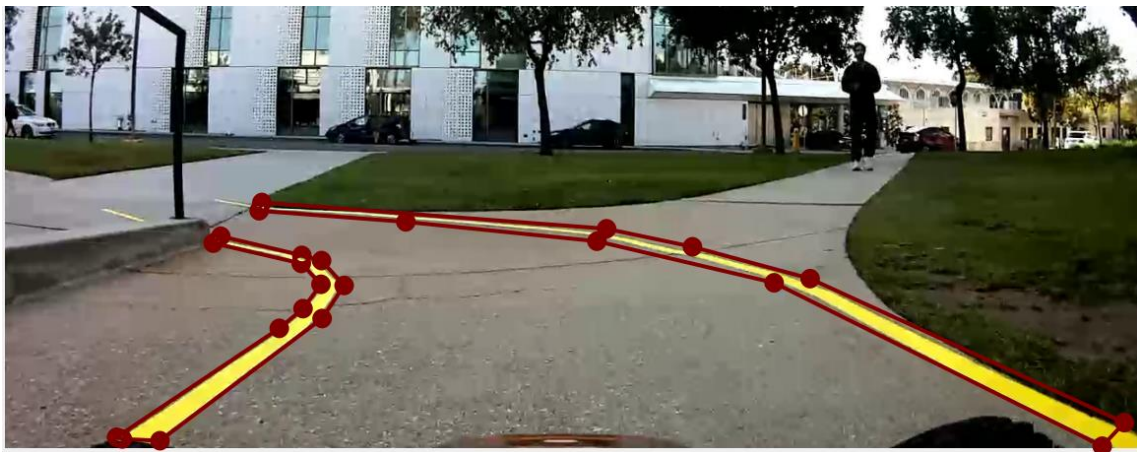


Figure 5: LabelMe Lane Annotation Used for U-Net Training

Next, the centerline points are transformed into a structured reference path in the vehicle's local coordinate frame. A target point is then selected using a lookahead strategy, allowing the controller to anticipate upcoming curvature and generate smoother steering behavior.



Figure 6: U-Net Lane Segmentation and Pure Pursuit Path Generation

The control stage uses the Pure Pursuit algorithm to continuously compute the steering angle required to minimize lateral error between the vehicle and the target point. Steering limits, rate limiting, and fallback behavior are also applied to improve system stability and prevent erratic motion when perception confidence is low.

Finally, the generated steering and throttle commands are transmitted over the CAN bus to the Arduino actuation system.

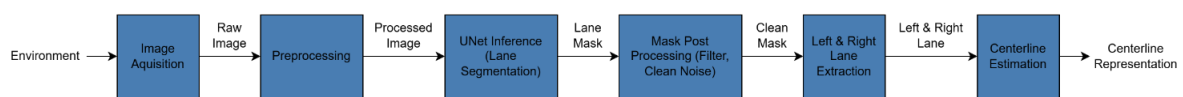


Figure 7: SMART Car Supreme Software Pipeline

3.3.3 Control Algorithm

The SMART Car Supreme uses the Pure Pursuit controller for trajectory tracking and lane-following control. Pure Pursuit is a geometric path-tracking algorithm that determines steering commands based on a target point located ahead of the vehicle along the desired path.

The controller computes the steering angle required to guide the vehicle toward the selected lookahead point while continuously updating the vehicle trajectory as new lane detections are processed. This approach provides smooth steering behavior and stable path tracking while remaining computationally efficient for embedded real-time systems.

To improve robustness, additional safety constraints are applied including steering saturation limits, rate limiting, and fallback stop behavior when lane confidence drops below a predefined threshold.

3.3.4 Work Breakdown Structure

A Work Breakdown Structure (WBS) was developed to organize project responsibilities across subsystem development areas including perception, control, electronics integration, communication, and testing.

The WBS provided a structured framework for task allocation, milestone tracking, and subsystem integration throughout the project lifecycle.

Figure 8 shows the breakdown of how work was split up for the project.

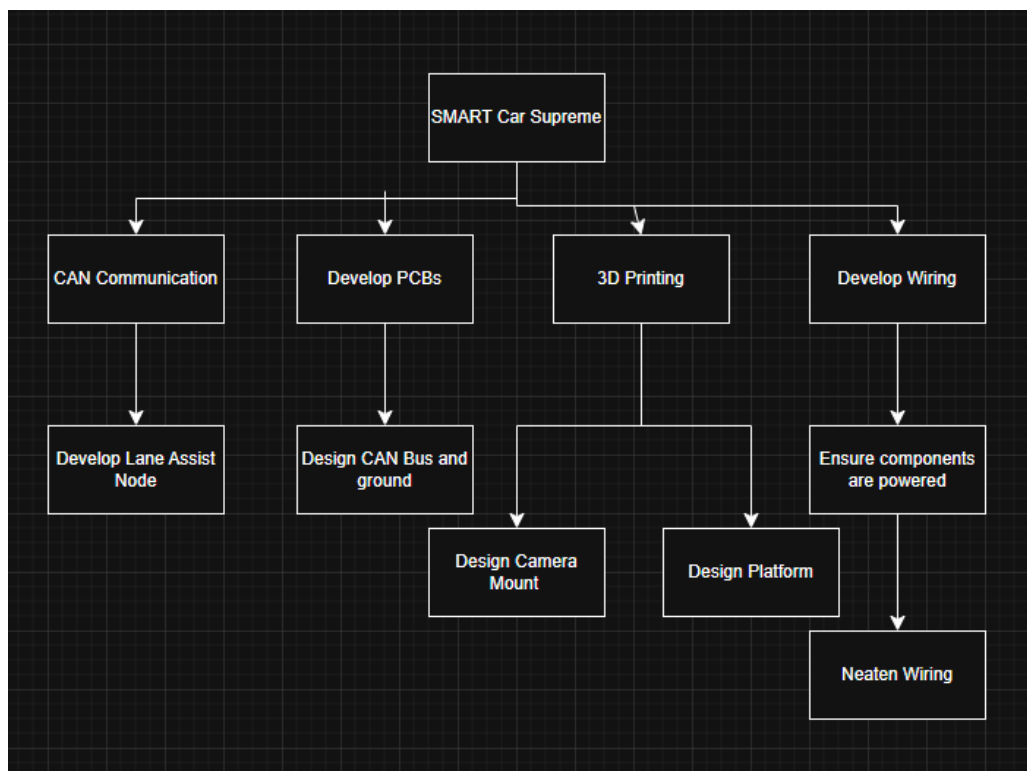


Figure 8: WBS

3.4 Standards and Constraints

The SMART Car Supreme incorporates several engineering standards to support reliable communication, safe autonomous operation, and maintainable system integration.

IEEE 1722 informs the handling of time-sensitive sensor and perception data, supporting low-latency communication for steering and braking responses. IEEE 21451 guides sensor integration and modular interfacing practices, helping standardize communication between sensing and control subsystems. IEEE 29119 provides general principles for software testing and verification, influencing unit testing, subsystem integration testing, and validation of autonomous behavior.

Additionally, principles from IEEE 1474 influenced the implementation of fail-safe operation, braking logic, and predictable control behavior during autonomous operation. Because the project utilizes custom PCBs, IEEE 1687 supports structured debugging, verification, and maintainability of embedded electronic systems.

The project also operates under several practical constraints, including:

Limited onboard processing power and battery capacity

Environmental variability such as glare, shadows, and uneven outdoor lighting

Sensor accuracy limitations

Safety requirements for operation in public spaces

Project budget and semester time constraints

These standards and constraints define the operational boundaries within which the SMART Car Supreme must reliably function.

3.5 Design Impact

The SMART Car Supreme has meaningful implications across environmental, societal, economic, and educational dimensions.

From an environmental perspective, the system promotes efficient electric propulsion and minimizes unnecessary energy consumption through controlled speed and responsive braking behavior. The transition from loose wiring to modular PCB-based integration also improves maintainability and reduces material waste over repeated project iterations.

Societally, the project contributes to research and education in autonomous mobility by demonstrating core concepts including lane following, collision avoidance, and predictable vehicle behavior. These capabilities mirror the foundational principles used in modern autonomous vehicle systems designed to reduce human error and improve transportation safety.

Economically, the project demonstrates how autonomous vehicle concepts can be implemented using relatively low-cost sensors, embedded hardware, open-source software, and student-designed electronics. The modular system architecture also lowers maintenance and future development costs by simplifying subsystem replacement and debugging.

Ethically, the SMART Car Supreme prioritizes safety-first system behavior through constrained operating speeds, fail-safe braking logic, structured testing, and transparent reporting of system limitations. The project reflects professional engineering responsibilities by emphasizing safe operation, maintainability, and responsible autonomous system design.

4 Electrical Design

4.1 Schematic

The SMART Car Supreme electrical system integrates perception, computation, and actuation subsystems into a centralized autonomous vehicle architecture. The system is powered through an onboard battery and regulated using dedicated power conversion hardware to provide stable operating voltages for the Jetson, Arduino, sensors, and communication modules.

The NVIDIA Jetson Orin Nano serves as the primary compute platform and interfaces with the camera and LiDAR sensors for perception processing. Steering and throttle commands generated by the Jetson are transmitted over the CAN bus to the Arduino microcontroller, which converts these commands into PWM signals for motor and steering actuation.

The schematic for the SMART Car Supreme is shown in Figure 9.

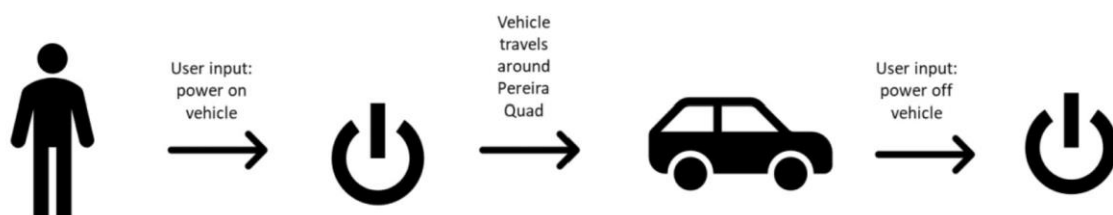


Figure 9: SMART Car Supreme System Schematic

4.2 Wiring and Cable Diagram

The wiring architecture was designed to improve organization, modularity, and maintainability compared to previous SMART Car iterations. Major components are mounted on a 3D-printed platform positioned above the battery system to simplify routing and reduce cable congestion.

The camera is mounted above the vehicle using a dedicated 3D-printed support structure to improve visibility and lane perception performance. The steering servo and drive motor remain mounted beneath the platform, while communication and power routing are distributed through dedicated PCB modules.

Figure 10 shows a block diagram of SMART Car Supreme. The components will mostly be placed on a 3D printed platform in order to keep everything organized and above the batteries. The camera is attached to the roof of the car via a 3D printed mount. The motor and servo are also below the platform.

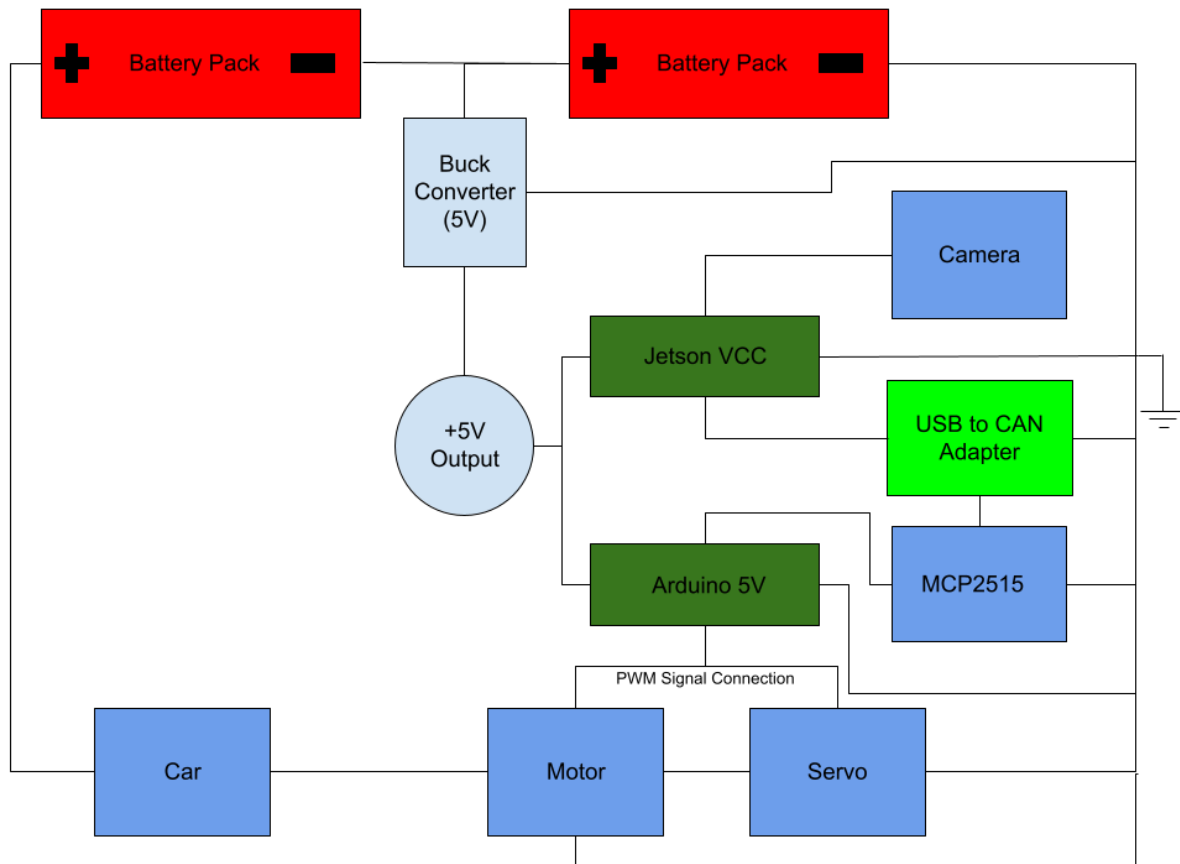


Figure 10: Block Diagram

4.3 Printed Circuit Board (PCB) Layout

Custom PCBs were developed to improve wiring reliability, simplify debugging, and support modular subsystem integration. The PCB architecture distributes power, supports CAN communication, and organizes sensor and actuator connections into structured interfaces.

Figure 11 shows the PCB designed to interface the Arduino microcontroller with the MCP2515 CAN controller. This board simplifies CAN integration and improves signal organization between control and communication hardware.

Figure 11 shows the PCB layout for the design intended to connect the Arduino and the MCP2515.

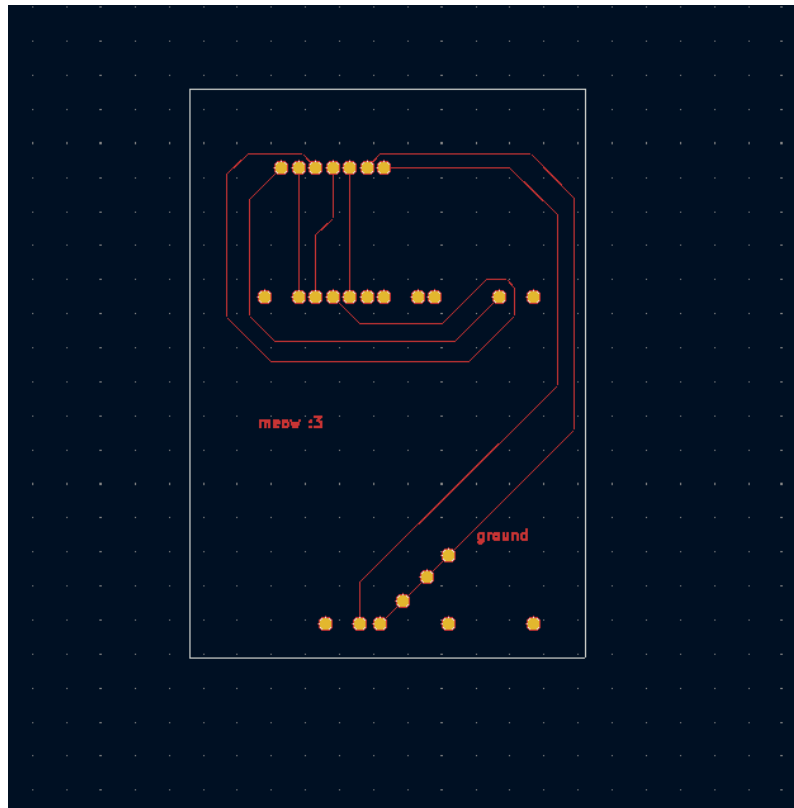


Figure 11: PCB layout for Arduino to MCP2515

Figure 12 shows the PCB used for CAN bus and ground distribution. This board centralizes communication and grounding connections, reducing wiring complexity and improving maintainability.

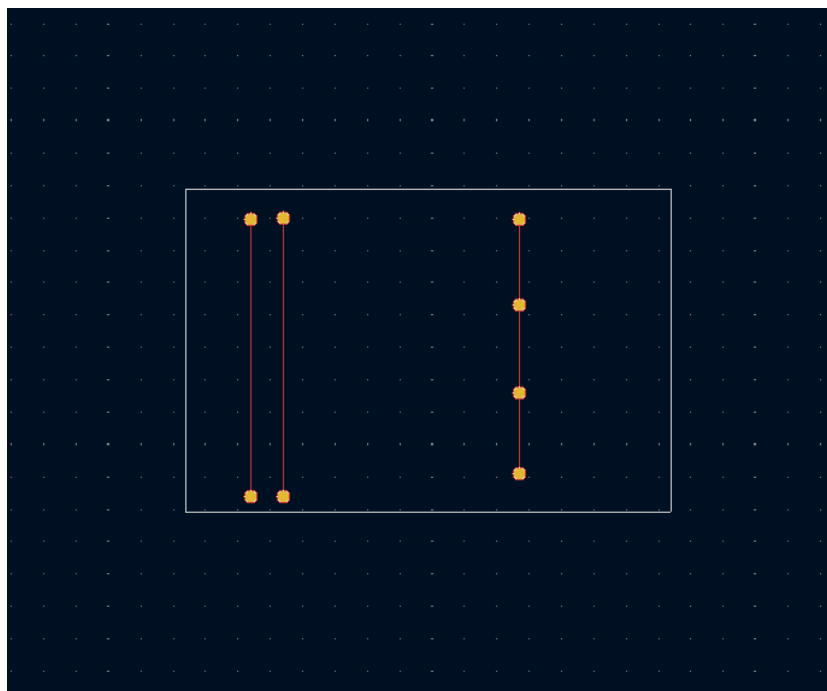


Figure 12: PCB Layout for CAN Bus and ground bus.

4.4 Bill of Materials

Table 17 summarizes the primary components purchased specifically for the SMART Car Supreme project. Components retained from previous SMART Car iterations are not included in this table.

Table 17: Bill of Materials

Item	Cost Per Part	Quantity	Total Cost
Camera	\$170	1	\$170
PWM	\$6	2	\$12
IC3 to IC5 Adaptor	\$6	2	\$12
IC3 Y-Splitter	\$15	2	\$30

4.5 Mechanical Images

Several custom 3D-printed components were developed to improve system organization, structural support, and sensor mounting.

Figure 13 shows the support leg used to elevate the electronics platform above the battery system and wiring harness. This separation improves cable routing and accessibility during maintenance.

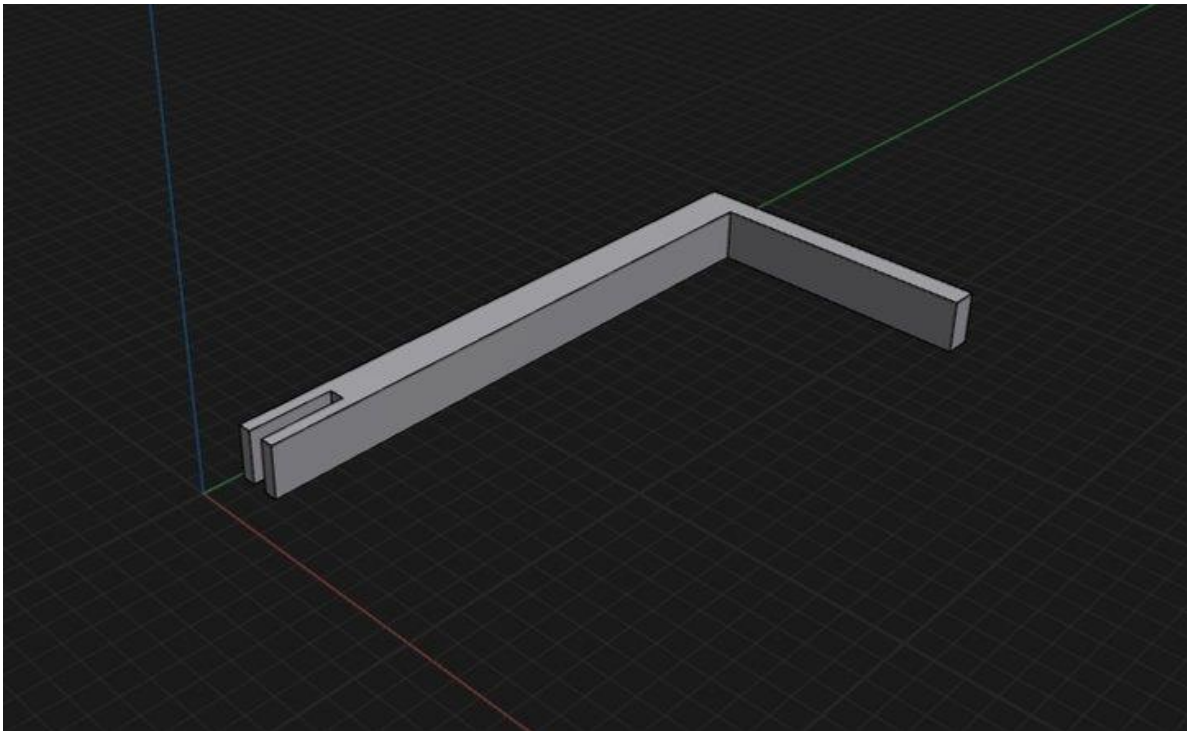


Figure 13: Support Leg for Platform

Figure 14 shows the camera mounting structure used to position the forward-facing camera above the vehicle for improved lane visibility and perception accuracy.

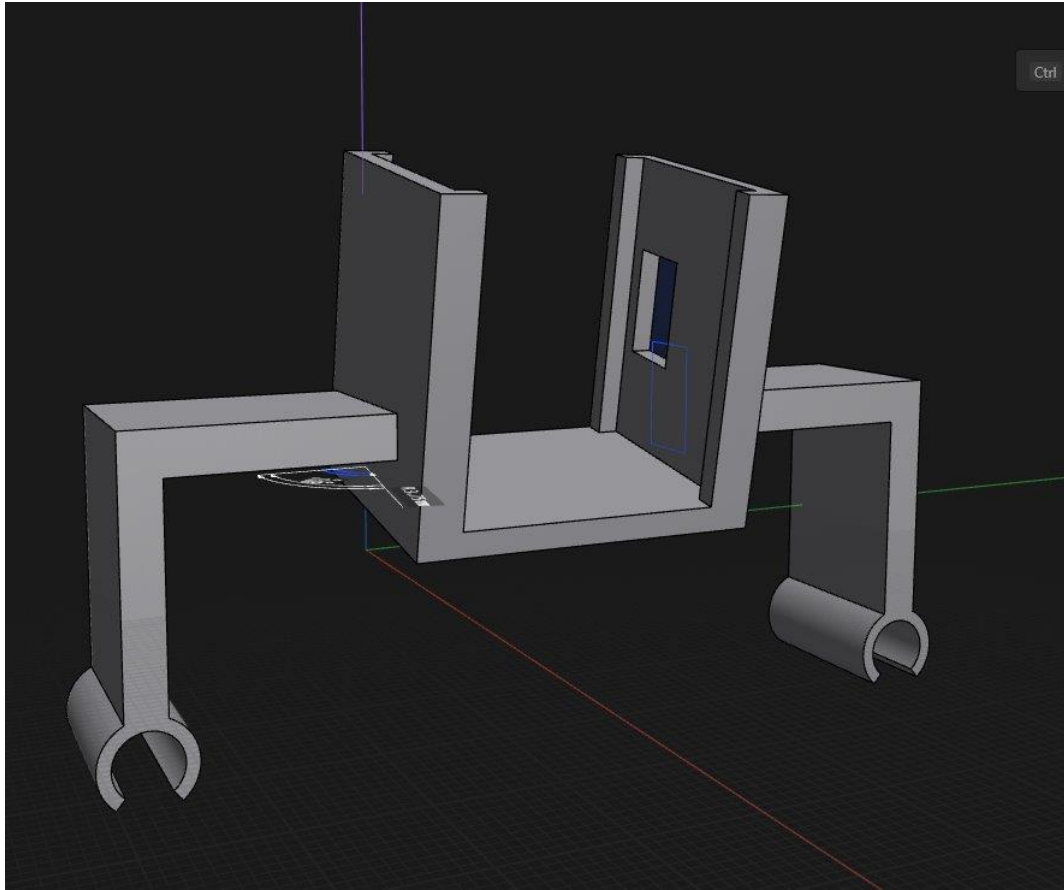


Figure 14: Camera Mount

Figure 15 shows one half of the modular electronics platform used to support system components above the battery assembly. The platform provides a stable mounting surface while improving organization and maintainability.

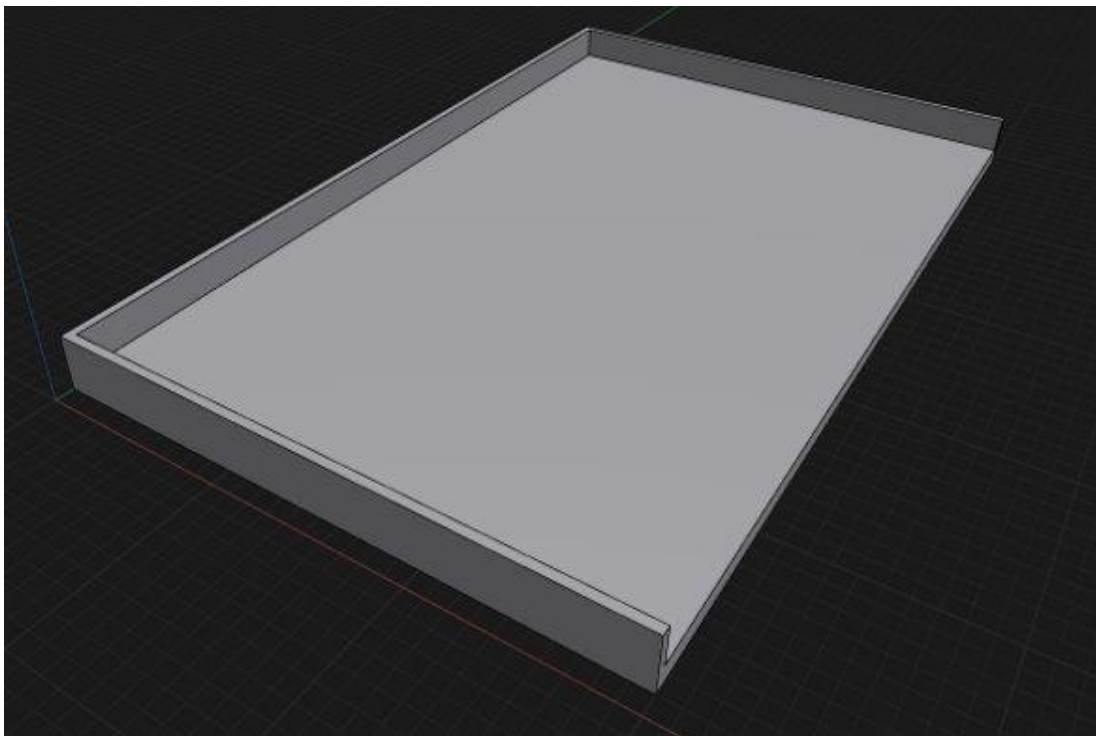


Figure 15: Platform for Components

4.6 System Design and Control Workflow

The SMART Car Supreme follows a real-time perception–planning–control pipeline to perform autonomous lane-following.

The system begins with lane detection using either simulation input or the forward-facing camera. The perception system generates centerline points representing the estimated lane geometry ahead of the vehicle.

These centerline points are converted into a structured reference path, from which a target point is selected using a lookahead strategy. The Pure Pursuit controller then computes the steering angle required to guide the vehicle toward the target point while minimizing lateral error.

Additional safety constraints, including steering limits, rate limiting, and fallback behavior, are applied before steering and throttle commands are converted into PWM signals and transmitted through the CAN bus to the vehicle actuators.

Finally, the system logs lane detections, control commands, timing information, and system states for debugging and performance evaluation.

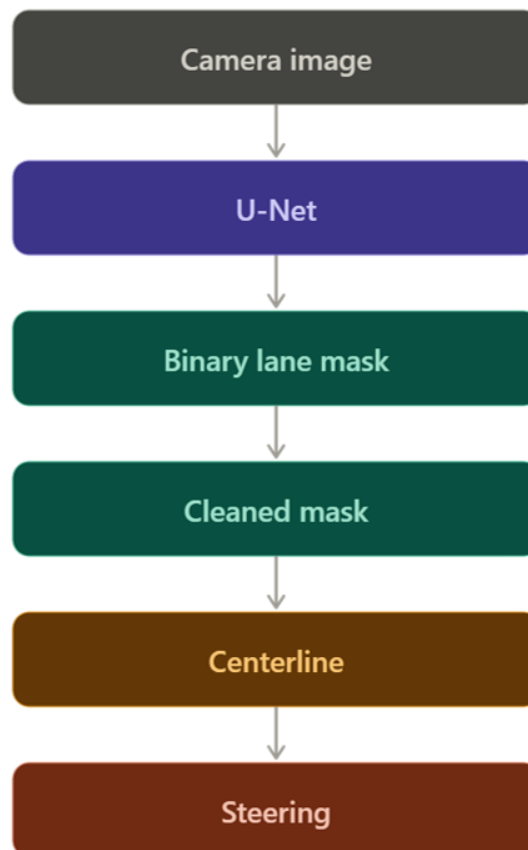


Figure 16: Autonomous Driving Software Pipeline

4.7 Cost Estimates

The total cost of the SMART Car Supreme project is summarized in Table 18. The complete system cost is approximately \$1,776, although many major components were reused from previous SMART Car iterations.

Table 198 Cost Estimates

Item	Cost Per Part	Quantity
Camera	\$170	1
PWM	\$6	2
IC3 to IC5 Adaptor	\$6	2
IC3 Y-Splitter	\$15	2
RC Car	\$1200	1
Jetson Orin Nano Developer Kit	\$250	1
Arduino Uno	\$20	1
MCP 2515	\$2	1
CAN Converter	\$30	1
Buck Converter	\$50	1
Total Cost	1,776	1

5 Experimental Test and Demonstration

5.1 Tests Used to Benchmark Performance

The Engineering requirements were listed in Table 11.

The engineering requirements defined in Section 3.2 were validated through subsystem testing, integrated system testing, and real-world autonomous driving demonstrations.

Lane-following performance was evaluated using the U-Net perception pipeline and Pure Pursuit controller operating on real-time camera input. Approximately 300 labeled image frames were used to train the U-Net model for lane segmentation. During testing, the perception system generated lane masks and computed centerline points used for steering control. Vehicle trajectory and lane position were visually monitored during repeated test laps to verify that the vehicle remained within the required lane boundaries.

Vehicle speed was validated by adjusting PWM throttle control to maintain a maximum operating speed of 3 ft/s. Speed performance was verified through timed driving tests over measured distances to ensure safe and stable operation.

Autonomous navigation capability was evaluated by testing the vehicle over repeated laps around Pereira Quad. The vehicle successfully completed multiple autonomous laps while maintaining stable trajectory tracking and control behavior.

Modular PCB integration and CAN communication were validated through subsystem testing using oscilloscope measurements and CAN communication diagnostics. The PCB successfully distributed CAN and grounding connections while demonstrating reliable communication between compute and actuation subsystems.

5.2 Working Prototype

The SMART Car Supreme prototype integrates perception, control, communication, and actuation into a functional autonomous vehicle system. The NVIDIA Jetson Orin Nano performs real-time lane detection and steering computation, while the Arduino microcontroller handles low-level PWM actuation for steering and throttle control.

The system is powered by a 14.8 V battery and regulated using a buck converter to provide stable voltage rails for onboard electronics. Camera input is processed through the U-Net perception pipeline to generate lane centerlines, which are then used by the Pure Pursuit controller to compute steering commands.

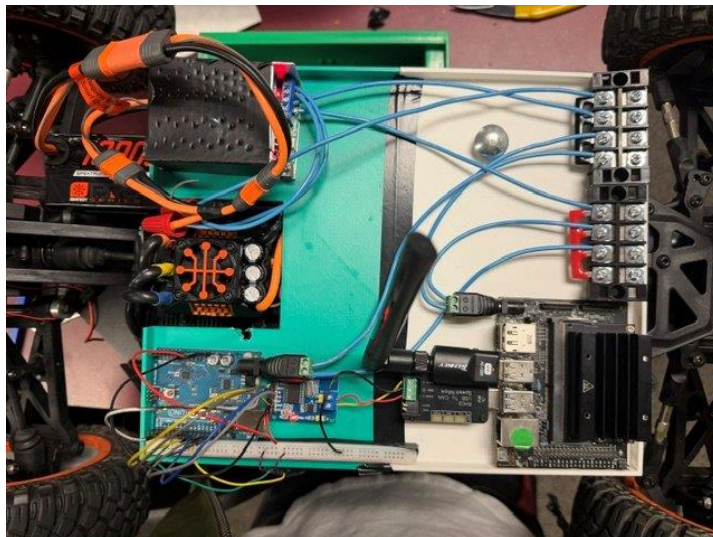


Figure 17: Integrated SMART Car Supreme Hardware Platform

Although the current implementation only required two CAN nodes, the system successfully demonstrated CAN-based communication using the MCP2515 interface. This validates the modular architecture for future expansion to additional sensing and control nodes.

Overall, the working prototype demonstrates reliable autonomous lane-following behavior and establishes a scalable platform for future improvements including obstacle detection, multi-sensor fusion, and dynamic speed control.

5.3 Demonstration

The SMART Car Supreme was demonstrated through a combination of subsystem validation and full-system autonomous testing.

At the system level, the vehicle successfully performed autonomous lane-following using real-time camera perception and onboard embedded processing. The Jetson platform continuously processed camera frames, generated lane centerlines, and computed steering commands using the Pure Pursuit controller. These commands were transmitted through the CAN bus to the Arduino, which converted them into PWM signals for motor and steering actuation.

At the hardware level, the PCB and CAN communication subsystems were validated using oscilloscope measurements and communication testing. CAN High and CAN Low signaling were verified, and the PCB demonstrated stable grounding and organized subsystem connectivity.

Additional demonstrations included manual steering and throttle validation to verify actuator

response and ensure reliable low-level control prior to autonomous operation.

These demonstrations verified that the perception, communication, control, and actuation subsystems functioned correctly both independently and as an integrated autonomous system.

5.4 Data Analytics

To evaluate system performance, both simulation and real-world testing data were analyzed. Key metrics included trajectory stability, steering response behavior, and qualitative lane-following accuracy.

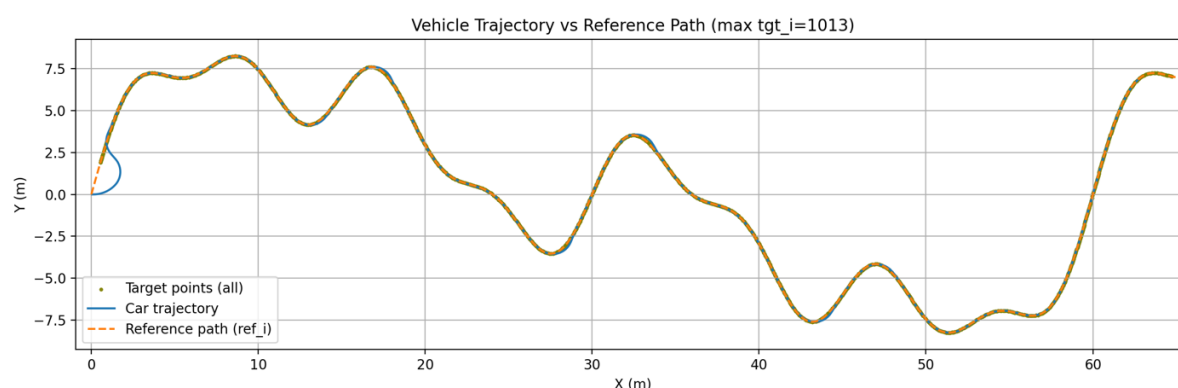


Figure 18: Autonomous Vehicle Trajectory and Controller Response

During testing, the vehicle demonstrated stable lane-following behavior at moderate operating speeds. However, oscillatory steering behavior was observed on straight segments when operating near the maximum speed limit. This behavior is attributed to system latency and the sensitivity of the Pure Pursuit controller to lookahead distance selection.

Environmental conditions such as glare, shadows, and inconsistent outdoor lighting introduced variability in lane detection confidence, occasionally affecting steering smoothness and control stability. Despite these challenges, the vehicle maintained overall lane stability and remained within the required operational constraints.

Future improvements will include quantitative logging of cross-track error, end-to-end latency measurement, and comparative evaluation of different controller configurations and lookahead strategies.

5.5 Customer Requirements

System testing confirmed that the SMART Car Supreme successfully met the primary customer requirements defined in Section 2.

The autonomous navigation requirement was satisfied through repeated successful laps around Pereira Quad using real-time perception and embedded control. Lane-following performance met the required ± 6 inch tolerance through the combined operation of the U-Net perception system and Pure Pursuit controller.

The system also satisfied the safety-related speed requirement by maintaining operation at or below the specified maximum speed of 3 ft/s. Stable low-speed operation improved control reliability and reduced stopping distance during testing.

Finally, the modular hardware requirement was partially satisfied through the implementation of PCB-based CAN and grounding distribution. While the current system configuration only required a limited number of nodes, the architecture supports future expansion to additional distributed sensing and control modules.

Overall, the SMART Car Supreme successfully demonstrated a reliable and modular autonomous vehicle platform capable of real-time lane-following and embedded system integration.

6 Ethics Consideration

The SMART Car Supreme project incorporates ethical considerations related to safety, transparency, reliability, and the broader societal impacts of autonomous systems. A primary ethical priority is ensuring safe and predictable vehicle behavior through the implementation of constrained operating speeds, structured control logic, and collision-avoidance mechanisms. These features reflect the engineering responsibility to prioritize public safety and minimize potential harm to users and bystanders.

System transparency and documentation are also important ethical considerations. Maintaining clear documentation of hardware configuration, software behavior, testing procedures, and system limitations supports reproducibility, accountability, and responsible engineering practice. Thorough validation and testing help ensure that autonomous behavior remains consistent and understandable during operation.

Although the SMART Car Supreme operates on a small academic scale, it reflects many of the ethical challenges associated with real-world autonomous vehicles. Autonomous systems raise broader societal questions regarding workforce displacement, reliance on automation, and decision-making without direct human intervention. While this project does not operate at commercial scale, it contributes to the ongoing discussion surrounding the responsible deployment of autonomous technologies.

Another important ethical consideration involves system behavior during unexpected or uncertain conditions. Autonomous systems must operate according to clearly defined safety constraints when rapid decisions are required without human input. The SMART Car Supreme addresses this through fail-safe behavior, constrained operating conditions, and control logic designed to prioritize safe system operation.

Overall, the project emphasizes responsible engineering practices by prioritizing safety, transparency, maintainability, and awareness of the broader societal implications of autonomous vehicle technology.

7 Contributions to ABET Program, LMU Values, Diversity, Social Community, Multidisciplinary, IEEE Values

7.1 ABET Program Contributions

The SMART Car Supreme project directly contributed to several ABET student learning outcomes through its emphasis on engineering design, embedded systems integration, experimentation, and collaborative development.

Outcome 1 – Engineering Problem Solving

The project required identifying and solving challenges related to autonomous navigation, lane detection reliability, modular hardware integration, and real-time embedded control.

Engineering principles were applied to evaluate sensing technologies, define measurable requirements, and develop a scalable autonomous system architecture.

Outcome 2 – Engineering Design

The project involved iterative, constraint-driven engineering design across perception, communication, control, and hardware integration subsystems. Trade studies involving sensing approaches, compute platforms, and modular PCB integration demonstrated the ability to design a system that balances performance, maintainability, scalability, and safety requirements.

Outcome 4 – Communication

Throughout the project, the team produced technical reports, design documentation, benchmarking analyses, and system diagrams to communicate engineering decisions and subsystem behavior. These deliverables supported design traceability and subsystem integration throughout development.

Outcome 5 – Teamwork

Successful completion of the project required coordination across sensing, control, software, and electronics integration tasks. Team members collaborated on subsystem implementation, testing, and debugging while managing shared project milestones and system integration challenges.

Outcome 6 – Experimentation and Data Analysis

The project required collecting, analyzing, and interpreting technical data related to lane detection performance, control behavior, communication reliability, and hardware integration. Experimental testing and benchmarking supported evidence-based engineering decisions throughout the design process.

Outcome 7 – Acquisition of New Knowledge

Development of CAN bus communication, Jetson-based perception pipelines, embedded control systems, and custom PCB integration required the team to independently learn new tools, frameworks, and engineering methodologies beyond prior coursework. Technical documentation, datasheets, and industry references were used extensively throughout development.

7.2 LMU Values

The SMART Car Supreme project reflects Loyola Marymount University's emphasis on academic excellence, ethical responsibility, and service-oriented engineering practice. The project challenged students to apply advanced concepts in embedded systems, control theory, sensing, and autonomous systems integration through hands-on engineering development.

The project also reflects LMU's commitment to ethical responsibility by emphasizing safe system behavior, transparent testing, and consideration of the societal implications of autonomous technologies. By prioritizing safety, maintainability, and responsible system design, the project aligns with LMU's goal of developing engineers who are both technically capable and socially responsible.

Additionally, the SMART Car Supreme serves as an educational platform that supports future student learning and multidisciplinary collaboration. The modular architecture and documented design process provide a foundation for future project teams to continue expanding the system and exploring advanced autonomous vehicle technologies.

7.3 IEEE Values

The SMART Car Supreme project aligns with IEEE’s mission of advancing technological innovation for the benefit of humanity through the development of safe, reliable, and maintainable autonomous systems.

The project reflects core IEEE ethical principles by prioritizing public safety, responsible system behavior, and transparent engineering practices. Safety-focused design decisions—including constrained operating speeds, fail-safe control behavior, structured testing procedures, and collision-avoidance logic—demonstrate a commitment to protecting users and bystanders during autonomous operation.

The project also incorporates engineering communication and embedded system integration principles consistent with industry practice, including CAN bus communication, SPI-based peripheral integration, and structured subsystem verification. These design approaches support reliability, modularity, and maintainability within the autonomous vehicle platform.

Overall, the SMART Car Supreme demonstrates the application of professional engineering standards and ethical system design principles in the development of an embedded autonomous vehicle system.

8 Conclusion

The SMART Car Supreme project successfully delivered a fully integrated 1/10-scale autonomous vehicle capable of real-time lane-following through a complete perception–control–actuation pipeline. By combining a camera-based U-Net lane detection system running on the NVIDIA Jetson Orin Nano with a Pure Pursuit controller and CAN-based communication to an Arduino actuator node, the system achieved reliable autonomous navigation under defined operating constraints.

A major accomplishment of the project was the successful demonstration of the vehicle autonomously completing full laps around Pereira Quad while remaining within lane boundaries. The perception pipeline consistently generated usable lane centerlines, allowing the controller to compute stable steering commands that maintained accurate trajectory tracking throughout operation. Across repeated test runs, the system satisfied the lane-following and navigation requirements established during system design and validation.

In addition to autonomous driving performance, the project introduced significant improvements in hardware organization, modularity, and system maintainability. The transition to PCB-based CAN communication and power distribution reduced wiring complexity and improved subsystem reliability compared to previous SMART Car iterations. Successful validation of CAN-based communication also demonstrated the feasibility of a modular distributed-control architecture for future system expansion.

Testing also revealed several practical limitations commonly encountered in real-world autonomous systems. Environmental conditions such as glare, shadows, and inconsistent outdoor lighting affected lane detection confidence, while minor steering oscillations were observed at higher operating speeds due to controller tuning sensitivity and system latency. These observations provide valuable insight for future improvements and continued platform development.

Overall, the SMART Car Supreme demonstrates a complete and functional embedded autonomous vehicle platform that successfully integrates perception, communication, control, and actuation into a reliable real-time system. The project establishes a strong foundation for future research and development of the SMART Car platform and provides a scalable architecture for continued autonomous vehicle experimentation.

9 Suggestions

Several opportunities exist for future improvement and expansion of the SMART Car Supreme platform.

Further PCB Integration

Future iterations should expand PCB integration to include additional subsystems such as sensor distribution, power regulation, and actuator routing. Increasing PCB modularity would further improve system organization, maintainability, and debugging efficiency.

Improved CAN Integration

Although CAN communication was successfully demonstrated, future versions should transition additional subsystems to distributed CAN nodes. Expanding CAN integration would improve scalability and better reflect modern automotive electronic architectures.

LiDAR Integration

Future development should fully integrate LiDAR-based obstacle detection into the autonomous driving pipeline. This would improve environmental awareness and enable more advanced autonomous behaviors beyond lane following alone.

Obstacle Avoidance

The current system primarily focuses on lane-following performance. Future iterations should implement dynamic obstacle avoidance and path replanning capabilities to allow the vehicle to safely navigate around obstacles rather than only stopping.

Dynamic Speed Control

The current implementation operates at a relatively constant speed for stability and safety. Future work should include adaptive speed control based on road curvature, obstacle proximity, and perception confidence to improve realism and overall autonomous performance.

Improved Perception Robustness

Additional work should focus on improving perception robustness under challenging outdoor conditions such as glare, shadows, and varying lighting environments. Potential improvements include expanded training datasets, image preprocessing techniques, and sensor fusion approaches.

Controller Optimization

Future testing should investigate alternative control strategies and lookahead tuning methods to reduce oscillatory behavior at higher speeds and improve trajectory smoothness during aggressive maneuvers.

10 References

- [1] K. Othman, "Exploring the implications of autonomous vehicles: a comprehensive review," *Innovative Infrastructure Solutions*, vol. 7, no. 2, Mar. 2022, doi: <https://doi.org/10.1007/s41062-022-00763-6> (accessed Sep. 28, 2025).
- [2] L. Hovhan, "From Level 0 to 5: The Steps to Autonomous Driving," Sapien.io, May 08, 2025. <https://www.sapien.io/blog/from-level-0-to-5-the-steps-to-autonomous-driving> (accessed Sep. 28, 2025).
- [3] "Waymo Driver," Waymo. <https://waymo.com/waymo-driver/> (accessed Sep. 28, 2025).
- [4] "Full Self Driving (Supervised)," Tesla. <https://www.tesla.com/fsd> (accessed Sep. 28, 2025).
- [5] "DRIVE PILOT," Mercedes-Benz USA. <https://www.mbusa.com/en/owners/manuals/drive-pilot#Technology> (accessed Sep. 28, 2025).
- [6] "Super Cruise 101: All you need to know about GM's hands-free driver assistance system," GM News, February 28, 2025. <https://news.gm.com/home.detail.html/Pages/topic/us/en/2025/feb/0228-supercruise.html> (accessed Sep. 28, 2025).

11 Appendices

11.1 Teammate Roles and Responsibilities

Project responsibilities were divided across software development, embedded systems integration, hardware design, and communication architecture development.

- **Joshua Oliveira and Brayden Kimura** focused primarily on software development, including lane detection, controller integration, perception pipelines, and autonomous driving logic.
- **Amelia Wirwick and Johnathan Yi** focused primarily on hardware integration, PCB development, and CAN bus communication setup.
- Integration and subsystem testing were performed collaboratively across the team to ensure successful communication between perception, control, and actuation subsystems.

11.2 Source Code and Supporting Files

Supporting materials for the SMART Car Supreme project include:

- Autonomous driving software
- U-Net lane detection implementation
- Pure Pursuit controller implementation
- CAN communication software
- PCB design files
- Testing and logging utilities

These materials support reproducibility, future development, and continued expansion of the SMART Car platform.

Codes

U Net Model:

```
import torch
import torch.nn as nn
class DoubleConv(nn.Module):
    def __init__(self, in_channels, out_channels):
        super().__init__()
        self.block = nn.Sequential(
            nn.Conv2d(in_channels, out_channels, 3, padding=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_channels, out_channels, 3, padding=1),
            nn.ReLU(inplace=True)
        )
    def forward(self, x):
        return self.block(x)
class UNet(nn.Module):
    def __init__(self, base_channels=32):
        super().__init__()
        c1 = base_channels
        c2 = base_channels * 2
        c3 = base_channels * 4
        c4 = base_channels * 8
        self.down1 = DoubleConv(3, c1)
        self.pool1 = nn.MaxPool2d(2)
        self.down2 = DoubleConv(c1, c2)
        self.pool2 = nn.MaxPool2d(2)
        self.down3 = DoubleConv(c2, c3)
        self.pool3 = nn.MaxPool2d(2)
        self.bottleneck = DoubleConv(c3, c4)
        self.up3 = nn.ConvTranspose2d(c4, c3, 2, stride=2)
        self.conv3 = DoubleConv(c3 * 2, c3)
        self.up2 = nn.ConvTranspose2d(c3, c2, 2, stride=2)
        self.conv2 = DoubleConv(c2 * 2, c2)
        self.up1 = nn.ConvTranspose2d(c2, c1, 2, stride=2)
        self.conv1 = DoubleConv(c1 * 2, c1)
        self.final = nn.Conv2d(c1, 1, kernel_size=1)
    def forward(self, x):
        d1 = self.down1(x)
        p1 = self.pool1(d1)
        d2 = self.down2(p1)
        p2 = self.pool2(d2)
        d3 = self.down3(p2)
        p3 = self.pool3(d3)
        bn = self.bottleneck(p3)
        u3 = self.up3(bn)
        u3 = torch.cat([u3, d3], dim=1)
        u3 = self.conv3(u3)
        u2 = self.up2(u3)
```



```

u2 = torch.cat([u2, d2], dim=1)
u2 = self.conv2(u2)
u1 = self.up1(u2)
u1 = torch.cat([u1, d1], dim=1)
u1 = self.conv1(u1)
return self.final(u1)

```

Training Model:

```

import os
import torch
from torch.utils.data import DataLoader
from dataset_loader import LaneDataset
from model import UNet
# -----
# settings
# -----
FRAMES_ROOT = "frames"
MASKS_ROOT = "masks"
IMAGE_SIZE = (256, 144)
CROP_TOP_FRACTION = 0.0
BATCH_SIZE = 4
NUM_EPOCHS = 15
VAL_RATIO = 0.2
SEED = 42
MODEL_BASE_CHANNELS = 32
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
# -----
# recursively gather image/mask pairs
# -----
samples = []
for subfolder in os.listdir(FRAMES_ROOT):
    frame_subdir = os.path.join(FRAMES_ROOT, subfolder)
    mask_subdir = os.path.join(MASKS_ROOT, subfolder)
    if not os.path.isdir(frame_subdir):
        continue
    if not os.path.isdir(mask_subdir):
        continue
    for file in os.listdir(mask_subdir):
        if not file.endswith(".png"):
            continue
        image_path = os.path.join(frame_subdir, file)
        mask_path = os.path.join(mask_subdir, file)
        if os.path.exists(image_path) and os.path.exists(mask_path):
            samples.append((image_path, mask_path))
    if len(samples) == 0:
        raise RuntimeError("No labeled image/mask pairs found. Run json_to_mask.py first.")
    print(f"Found {len(samples)} labeled pairs.")
# -----
# split samples + datasets
# -----
torch.manual_seed(SEED)
val_size = max(1, int(len(samples) * VAL_RATIO))
train_size = len(samples) - val_size

```

```

indices = torch.randperm(len(samples), generator=torch.Generator().manual_seed(SEED)).tolist()
train_samples = [samples[i] for i in indices[:train_size]]
val_samples = [samples[i] for i in indices[train_size:]]
train_dataset = LaneDataset(
    train_samples,
    image_size=IMAGE_SIZE,
    crop_top_fraction=CROP_TOP_FRACTION,
    train=True,
)
val_dataset = LaneDataset(
    val_samples,
    image_size=IMAGE_SIZE,
    crop_top_fraction=CROP_TOP_FRACTION,
    train=False,
)
train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False)
print(f"Train: {len(train_dataset)}")
print(f"Val: {len(val_dataset)}")
# -----
# model
# -----
model = UNet(base_channels=MODEL_BASE_CHANNELS).to(device)
criterion = torch.nn.BCEWithLogitsLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
# -----
# training loop
# -----
for epoch in range(NUM_EPOCHS):
    model.train()
    train_loss = 0.0
    for images, masks in train_loader:
        images = images.to(device)
        masks = masks.to(device)
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, masks)
        loss.backward()
        optimizer.step()
    train_loss += loss.item()
    model.eval()
    val_loss = 0.0
    with torch.no_grad():
        for images, masks in val_loader:
            images = images.to(device)
            masks = masks.to(device)
            outputs = model(images)
            loss = criterion(outputs, masks)
            val_loss += loss.item()
    print(
        f"Epoch {epoch+1}/{NUM_EPOCHS} | "
        f"Train Loss: {train_loss/len(train_loader):.4f} | "
        f"Val Loss: {val_loss/len(val_loader):.4f}"
    )
torch.save(model.state_dict(), "lane_unet_small.pth")

```

```
print("Model saved as lane_unet_small.pth")
```

Centerline

Extraction:

```
import time
import torch
import cv2
import numpy as np
from dataset_loader import IGNORE_TOP_FRACTION, inference_preprocess, mask_top_region
from model import UNet
from tensorrt_runner import TensorRTRunner
class LaneCenterlineDetector:
def __init__(
self,
model_path="lane_unet.pth",
threshold=0.3,
input_size=(256, 144),
crop_top_fraction=0.0,
target_x_bias_px=40,
device=None,
use_half=True,
verbose=True,
model_base_channels=32,
num_sample_rows=28,
):
self.model_path = model_path
self.threshold = threshold
self.input_size = input_size
self.crop_top_fraction = crop_top_fraction
self.target_x_bias_px = target_x_bias_px
self.verbose = verbose
self.model_base_channels = model_base_channels
self.ignore_top_fraction = IGNORE_TOP_FRACTION
if device is None:
self.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
else:
self.device = device
self.is_cuda = str(self.device).startswith("cuda")
self.use_half = bool(use_half and self.is_cuda)
if self.is_cuda:
torch.backends.cudnn.benchmark = True
# centerline settings
self.NUM_SAMPLE_ROWS = num_sample_rows
# Keep centerline extraction in the lower part of the frame so
# lookahead stays near the car instead of climbing too far uphill.
self.ROW_TOP_FRAC = 0.62
self.ROW_BOTTOM_FRAC = 0.90
self.MIN_COMPONENT_AREA = 120
self.MIN_BLOB_WIDTH = 3
self.MIN_COMPONENT_HEIGHT = 10
self.MIN_COMPONENT_BOTTOM_FRAC = 0.58
self.MAX_COMPONENT_WIDTH_TO_HEIGHT = 2.6
self.MAX_FLAT_COMPONENT_FILL_RATIO = 0.55
# row-to-row tracking
```

```

self.SEARCH_MARGIN_FRAC = 0.18
self.MAX_ROW_JUMP_FRAC = 0.18
self.DEFAULT_HALF_LANE_WIDTH_FRAC = 0.3
self.LANE_WIDTH_SMOOTHING = 0.20
self.CENTER_SMOOTHING = 0.35
# lane width sanity
self.MIN_LANE_WIDTH_FRAC = 0.15
self.MAX_LANE_WIDTH_FRAC = 0.80
# smoothing
self.POINT_SMOOTH_WINDOW = 5
self.TARGET_ALPHA = 0.20
# Choose a slightly nearer centerline sample so the target marker sits
# lower in the image and better matches the near-term driving corridor.
self.TARGET_INDEX_FAR_FRAC = 0.40
self.TARGET_INDEX_NEAR_FRAC = 0.18
self.SINGLE_LANE_TARGET_INDEX_FAR_FRAC = 0.32
self.SINGLE_LANE_TARGET_INDEX_NEAR_FRAC = 0.14
self.LOOKAHEAD_TURN_TRIGGER_FRAC = 0.50
self.TARGET_Y_VISUAL_OFFSET_PX = 50
self.SINGLE_LANE_CURVE_TRIGGER_PX = 4
self.SINGLE_LANE_TARGET_BIAS_GAIN = 4.0
self.SINGLE_LANE_TARGET_BIAS_MAX_FRAC = 0.75
# persistent target smoothing for video/live use
self.prev_target_x = None
self.prev_target_y = None
self.prev_frame_left_x = None
self.prev_frame_right_x = None
# cache morphology kernels
self.kernel_open = np.ones((3, 3), np.uint8)
self.kernel_close = np.ones((5, 5), np.uint8)
self.sample_rows = np.linspace(
    int(self.input_size[1] * self.ROW_BOTTOM_FRAC),
    int(self.input_size[1] * self.ROW_TOP_FRAC),
    self.NUM_SAMPLE_ROWS
).astype(int)
# model
self.backend = "tensorrt" if self.model_path.endswith(".engine") else "torch"
self.model = None
self.trt_runner = None
if self.backend == "tensorrt":
    self.trt_runner = TensorRTRunner(self.model_path, verbose=self.verbose)
else:
    self.model = UNet(base_channels=self.model_base_channels).to(self.device)
    if self.is_cuda:
        self.model = self.model.to(memory_format=torch.channels_last)
    try:
        state_dict = torch.load(self.model_path, map_location=self.device, weights_only=True)
    except TypeError:
        state_dict = torch.load(self.model_path, map_location=self.device)
    self.model.load_state_dict(state_dict)
    self.model.eval()
    if self.use_half:
        self.model.half()
    if self.verbose:
        print("CUDA available:", torch.cuda.is_available())

```

```

print("Detector device:", self.device)
print("Using FP16:", self.use_half)
print("Backend:", self.backend)
if self.model is not None:
print("Model parameter device:", next(self.model.parameters()).device)
# -----
# helpers
# -----
def largest_components(self, mask: np.ndarray, keep_n: int = 4, min_area: int = 120) -> np.ndarray:
num_labels, labels, stats, _ = cv2.connectedComponentsWithStats(mask, connectivity=8)
h, _ = mask.shape
min_bottom = int(round(h * self.MIN_COMPONENT_BOTTOM_FRAC))
comps = []
for i in range(1, num_labels):
area = stats[i, cv2.CC_STAT_AREA]
if area < min_area:
continue
top = stats[i, cv2.CC_STAT_TOP]
width = stats[i, cv2.CC_STAT_WIDTH]
height = stats[i, cv2.CC_STAT_HEIGHT]
bottom = top + height
if height < self.MIN_COMPONENT_HEIGHT:
continue
# Reject blobs that live too high in the crop, since the sampled
# driving corridor is taken from the lower-middle of the frame.
if bottom < min_bottom:
continue
# Grass and curb false positives often form broad, filled-in patches.
# On turns, a real outer lane edge can look almost horizontal in the
# image, so we only reject "flat" blobs when they are also dense.
fill_ratio = area / float(max(width * height, 1))
if (
width > height * self.MAX_COMPONENT_WIDTH_TO_HEIGHT
and fill_ratio > self.MAX_FLAT_COMPONENT_FILL_RATIO
):
continue
comps.append((area, i))
comps.sort(reverse=True)
cleaned = np.zeros_like(mask)
for _, idx in comps[:keep_n]:
cleaned[labels == idx] = 255
return cleaned
def postprocess_mask(self, mask: np.ndarray) -> np.ndarray:
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, self.kernel_open)
mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, self.kernel_close)
mask = self.largest_components(mask, keep_n=4, min_area=self.MIN_COMPONENT_AREA)
return mask
def get_row_blobs(self, mask_row: np.ndarray):
xs = np.where(mask_row > 0)[0]
if len(xs) == 0:
return []
blobs = []
start = xs[0]
prev = xs[0]
for x in xs[1:]:

```

```

if x == prev + 1:
    prev = x
else:
    left = start
    right = prev
    width = right - left + 1
    if width >= self.MIN_BLOB_WIDTH:
        blobs.append({
            "left": int(left),
            "right": int(right),
            "center": int((left + right) // 2),
            "width": int(width)
        })
    start = x
    prev = x
    left = start
    right = prev
    width = right - left + 1
    if width >= self.MIN_BLOB_WIDTH:
        blobs.append({
            "left": int(left),
            "right": int(right),
            "center": int((left + right) // 2),
            "width": int(width)
        })
    return blobs
def choose_blob_near_target(self, blobs, target_x, margin):
    if target_x is None or len(blobs) == 0:
        return None
    candidates = []
    for b in blobs:
        dist = abs(b["center"] - target_x)
        if dist <= margin:
            candidates.append((dist, -b["width"], b))
    if len(candidates) == 0:
        return None
    candidates.sort(key=lambda t: (t[0], t[1]))
    return candidates[0][2]
def is_candidate_plausible(self, candidate_x, prev_x, max_jump):
    if candidate_x is None or prev_x is None:
        return True
    return abs(candidate_x - prev_x) <= max_jump
def choose_extreme_blob(self, blobs, side, center_x):
    if len(blobs) == 0:
        return None
    if side == "left":
        left_blobs = [b for b in blobs if b["center"] < center_x]
        if len(left_blobs) == 0:
            return None
        left_blobs.sort(key=lambda b: b["center"], reverse=True)
        return left_blobs[0]
    right_blobs = [b for b in blobs if b["center"] >= center_x]
    if len(right_blobs) == 0:
        return None
    return right_blobs.sort(key=lambda b: b["center"])

```

```

return right_blobs[0]
def assign_single_blob_side(self, blob, prev_left_x, prev_right_x, img_center_x):
    center = blob["center"]
    if prev_left_x is not None and prev_right_x is not None:
        dist_left = abs(center - prev_left_x)
        dist_right = abs(center - prev_right_x)
        return "left" if dist_left <= dist_right else "right"
    if prev_left_x is not None:
        return "left"
    if prev_right_x is not None:
        return "right"
    return "left" if center < img_center_x else "right"
def find_lane_positions_in_row(self, mask_row: np.ndarray, prev_left_x, prev_right_x, prev_center_x,
    half_lane_width):
    w = len(mask_row)
    img_center_x = w // 2
    search_margin = int(w * self.SEARCH_MARGIN_FRAC)
    max_row_jump = int(w * self.MAX_ROW_JUMP_FRAC)
    blobs = self.get_row_blobs(mask_row)
    if len(blobs) == 0:
        return None, None
    if len(blobs) == 1:
        blob = blobs[0]
        side = self.assign_single_blob_side(blob, prev_left_x, prev_right_x, img_center_x)
        if side == "left":
            return blob["center"], None
        return None, blob["center"]
    left_blob = None
    right_blob = None
    if prev_left_x is not None:
        left_blob = self.choose_blob_near_target(blobs, prev_left_x, search_margin)
    if prev_right_x is not None:
        right_blob = self.choose_blob_near_target(blobs, prev_right_x, search_margin)
    if left_blob is not None and right_blob is not None and left_blob["center"] == right_blob["center"]:
        if prev_center_x is None:
            prev_center_x = img_center_x
        if left_blob["center"] < prev_center_x:
            right_blob = None
        else:
            left_blob = None
    split_center = prev_center_x if prev_center_x is not None else img_center_x
    if left_blob is None:
        left_blob = self.choose_extreme_blob(blobs, "left", split_center)
    if right_blob is None:
        right_blob = self.choose_extreme_blob(blobs, "right", split_center)
    left_x = left_blob["center"] if left_blob is not None else None
    right_x = right_blob["center"] if right_blob is not None else None
    if not self.is_candidate_plausible(left_x, prev_left_x, max_row_jump):
        left_x = None
    if not self.is_candidate_plausible(right_x, prev_right_x, max_row_jump):
        right_x = None
    if left_x is None and prev_left_x is not None:
        left_blob = self.choose_blob_near_target(blobs, prev_left_x, max_row_jump)
        left_x = left_blob["center"] if left_blob is not None else None
    if right_x is None and prev_right_x is not None:

```

```

right_blob = self.choose_blob_near_target(blobs, prev_right_x, max_row_jump)
right_x = right_blob["center"] if right_blob is not None else None
if left_x is not None and right_x is not None and right_x <= left_x:
    if prev_center_x is not None:
        if abs(left_x - (prev_center_x - half_lane_width)) < abs(right_x - (prev_center_x + half_lane_width)):
            right_x = None
        else:
            left_x = None
    else:
        left_x, right_x = None, None
if left_x is not None and right_x is not None:
    lane_width = right_x - left_x
    min_lane_width = int(w * self.MIN_LANE_WIDTH_FRAC)
    max_lane_width = int(w * self.MAX_LANE_WIDTH_FRAC)
    if lane_width < min_lane_width or lane_width > max_lane_width:
        expected_left = prev_center_x - half_lane_width if prev_center_x is not None else None
        expected_right = prev_center_x + half_lane_width if prev_center_x is not None else None
        left_err = abs(left_x - expected_left) if expected_left is not None else float("inf")
        right_err = abs(right_x - expected_right) if expected_right is not None else float("inf")
        if left_err <= right_err:
            right_x = None
        else:
            left_x = None
return left_x, right_x
def smooth_points(self, points, window=5, use_poly=False, poly_deg=2):
    """
    Smooth a list of (x, y) points along the x axis.
    use_poly=True → fit a degree-`poly_deg` polynomial y->x and resample.
    Best for the centerline: eliminates squiggles caused by
    row-to-row blob mis-assignment.
    use_poly=False → original moving-average (kept for left/right edges).
    """
    if len(points) < 3:
        return points
    xs = np.array([p[0] for p in points], dtype=np.float32)
    ys = np.array([p[1] for p in points], dtype=np.float32)
    if use_poly:
        # Need at least poly_deg+1 points for a valid fit
        if len(points) < poly_deg + 1:
            return points
        try:
            coeffs = np.polyfit(ys, xs, poly_deg)
            xs_smooth = np.polyval(coeffs, ys)
        except (np.linalg.LinAlgError, ValueError):
            # Fall back to moving average if fit fails
            pass
        else:
            return [(int(round(float(x))), int(y)) for x, y in zip(xs_smooth, ys)]
    # --- original moving-average path ---
    if window < 3:
        return points
    if len(points) < window:
        window = len(points) if len(points) % 2 == 1 else len(points) - 1
    if window < 3:
        return points

```



```

pad = window // 2
xs_padded = np.pad(xs, (pad, pad), mode="edge")
kernel = np.ones(window, dtype=np.float32) / window
xs_smooth = np.convolve(xs_padded, kernel, mode="valid")
return [(int(round(float(x))), int(y)) for x, y in zip(xs_smooth, ys)]
def extract_centerline(self, mask: np.ndarray, initial_left_x=None, initial_right_x=None, initial_center_x=None):
    h, w = mask.shape
    if h == self.input_size[1]:
        rows = self.sample_rows
    else:
        rows = np.linspace(
            int(h * self.ROW_BOTTOM_FRAC),
            int(h * self.ROW_TOP_FRAC),
            self.NUM_SAMPLE_ROWS
        ).astype(int)
    half_lane_width = int(w * self.DEFAULT_HALF_LANE_WIDTH_FRAC)
    centers = []
    prev_left_x = initial_left_x
    prev_right_x = initial_right_x
    prev_center_x = initial_center_x if initial_center_x is not None else w // 2
    left_points = []
    right_points = []
    min_lane_width = int(w * self.MIN_LANE_WIDTH_FRAC)
    max_lane_width = int(w * self.MAX_LANE_WIDTH_FRAC)
    # Tightened from 0.35 → 0.15 so noisy rows contribute less before the
    # poly fit cleans up the full centerline at the end.
    center_smoothing = 0.15
    for y in rows:
        row = mask[y, :]
        left_x, right_x = self.find_lane_positions_in_row(
            row,
            prev_left_x=prev_left_x,
            prev_right_x=prev_right_x,
            prev_center_x=prev_center_x,
            half_lane_width=half_lane_width
        )
        center_x = None
        if left_x is not None and right_x is not None and right_x > left_x:
            lane_width = right_x - left_x
            if min_lane_width <= lane_width <= max_lane_width:
                measured_half = lane_width // 2
                half_lane_width = int(
                    (1.0 - self.LANE_WIDTH_SMOOTHING) * half_lane_width +
                    self.LANE_WIDTH_SMOOTHING * measured_half
                )
                center_x = (left_x + right_x) // 2
            else:
                left_x = None
                right_x = None
        if center_x is None and left_x is not None:
            center_x = left_x + half_lane_width
        elif center_x is None and right_x is not None:
            center_x = right_x - half_lane_width
        elif center_x is None:
            center_x = prev_center_x

```

```

# In single-lane turns, prevent the inferred centerline from reversing
# direction row-to-row and forming an S-shaped path.
if prev_center_x is not None:
    if left_x is None and right_x is not None:
        right_dx = right_x - prev_right_x if prev_right_x is not None else 0
        if right_dx < -self.SINGLE_LANE_CURVE_TRIGGER_PX:
            center_x = min(center_x, prev_center_x)
        elif right_dx > self.SINGLE_LANE_CURVE_TRIGGER_PX:
            center_x = max(center_x, prev_center_x)
        elif right_x is None and left_x is not None:
            left_dx = left_x - prev_left_x if prev_left_x is not None else 0
            if left_dx > self.SINGLE_LANE_CURVE_TRIGGER_PX:
                center_x = max(center_x, prev_center_x)
            elif left_dx < -self.SINGLE_LANE_CURVE_TRIGGER_PX:
                center_x = min(center_x, prev_center_x)
        # When both edges are visible, trust the measured midpoint.
        # Row-to-row smoothing is mainly useful for inferred single-lane
        # centers; on sharp turns it can make the upper centerline lag and
        # cut across the corridor.
        if prev_center_x is not None and not (left_x is not None and right_x is not None):
            center_x = int(
                (1.0 - center_smoothing) * prev_center_x +
                center_smoothing * center_x
            )
        center_x = max(0, min(w - 1, center_x))
        centers.append((center_x, y))
        if left_x is not None:
            left_points.append((left_x, y))
            prev_left_x = left_x
        else:
            prev_left_x = max(0, center_x - half_lane_width)
        if right_x is not None:
            right_points.append((right_x, y))
            prev_right_x = right_x
        else:
            prev_right_x = min(w - 1, center_x + half_lane_width)
        prev_center_x = center_x
        left_points = self.smooth_points(left_points, window=self.POINT_SMOOTH_WINDOW, use_poly=False)
        right_points = self.smooth_points(right_points, window=self.POINT_SMOOTH_WINDOW, use_poly=False)
        # Centers get the poly fit — this is the key change that kills the squiggle
        # Keep center smoothing local so sharp turns are not rounded into a
        # single global curve that can leave the lane corridor.
        centers = self.smooth_points(centers, window=self.POINT_SMOOTH_WINDOW, use_poly=False)
    return centers, left_points, right_points, half_lane_width

def draw_polyline(self, image: np.ndarray, points, color, thickness=3):
    out = image.copy()
    if len(points) >= 2:
        pts = np.array(points, dtype=np.int32).reshape((-1, 1, 2))
        cv2.polylines(out, [pts], False, color, thickness)
    elif len(points) == 1:
        cv2.circle(out, points[0], 4, color, -1)
    return out

def draw_debug_rows(self, image: np.ndarray, centers, left_points, right_points):
    out = image.copy()
    for x, y in centers:

```

```

cv2.circle(out, (x, y), 4, (0, 255, 255), -1)
for x, y in left_points:
cv2.circle(out, (x, y), 3, (255, 0, 255), -1)
for x, y in right_points:
cv2.circle(out, (x, y), 3, (255, 165, 0), -1)
return out
def reset_tracking(self):
self.prev_target_x = None
self.prev_target_y = None
self.prev_frame_left_x = None
self.prev_frame_right_x = None
# -----
# fast internals
# -----
def _prepare_input(self, frame_crop: np.ndarray):
image = cv2.resize(frame_crop, self.input_size, interpolation=cv2.INTER_LINEAR)
image = inference_preprocess(image)
image = mask_top_region(image, self.ignore_top_fraction)
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
image = image.astype(np.float32) / 255.0
image = np.transpose(image, (2, 0, 1))
return np.ascontiguousarray(np.expand_dims(image, axis=0))
def _predict_mask(self, frame_crop: np.ndarray):
model_input = self._prepare_input(frame_crop)
if self.backend == "tensorrt":
output = self.trt_runner.infer(model_input)
output = 1.0 / (1.0 + np.exp(-output))
output = output.squeeze().astype(np.float32, copy=False)
else:
tensor = torch.from_numpy(model_input)
if self.is_cuda:
tensor = tensor.contiguous(memory_format=torch.channels_last)
if self.use_half:
tensor = tensor.half()
tensor = tensor.to(self.device, non_blocking=True)
with torch.inference_mode():
output = self.model(tensor)
output = torch.sigmoid(output)
output = output.squeeze().float().cpu().numpy()
raw_mask = (output * 255).astype(np.uint8)
mask = (output > self.threshold).astype(np.uint8) * 255
mask = mask_top_region(mask, self.ignore_top_fraction)
raw_mask = mask_top_region(raw_mask, self.ignore_top_fraction)
return raw_mask, mask
def _scale_points(self, points, scale_x: float, scale_y: float):
if not points:
return []
return [
(
int(round(x * scale_x)),
int(round(y * scale_y)),
)
for x, y in points
]
def _scale_point(self, point, scale_x: float, scale_y: float):

```

```

if point is None:
    return None
x, y = point
return int(round(x * scale_x)), int(round(y * scale_y))
def _single_lane_target_bias(self, left_points_small, right_points_small, half_lane_width_small: int, mask_width: int)
-> int:
    """
    Bias the target harder into a turn when only the outer lane edge remains visible.
    Example:
    - right turn: left lane stays visible, right lane disappears -> bias target right
    - left turn: right lane stays visible, left lane disappears -> bias target left
    """
    if len(left_points_small) >= 2 and len(right_points_small) == 0:
        bottom_x = left_points_small[0][0]
        top_x = left_points_small[-1][0]
        curve_dx = top_x - bottom_x
        if curve_dx > self.SINGLE_LANE_CURVE_TRIGGER_PX:
            raw_bias = int(round(curve_dx * self.SINGLE_LANE_TARGET_BIAS_GAIN))
            max_bias = int(round(mask_width * self.SINGLE_LANE_TARGET_BIAS_MAX_FRAC))
            return max(0, min(raw_bias, max_bias))
        if len(right_points_small) >= 2 and len(left_points_small) == 0:
            bottom_x = right_points_small[0][0]
            top_x = right_points_small[-1][0]
            curve_dx = top_x - bottom_x
            if curve_dx < -self.SINGLE_LANE_CURVE_TRIGGER_PX:
                raw_bias = int(round((-curve_dx) * self.SINGLE_LANE_TARGET_BIAS_GAIN))
                max_bias = int(round(mask_width * self.SINGLE_LANE_TARGET_BIAS_MAX_FRAC))
                return -max(0, min(raw_bias, max_bias))
            return 0
    def _single_lane_turn_direction(self, left_points_small, right_points_small):
        # When only one lane edge remains visible, treat the missing side as the
        # inner lane and command a hard turn in that direction.
        if len(right_points_small) >= 1 and len(left_points_small) == 0:
            return "left"
        if len(left_points_small) >= 1 and len(right_points_small) == 0:
            return "right"
        return None
    def _target_index(self, centers_small, left_points_small, right_points_small) -> int:
        if len(centers_small) == 0:
            return 0
        xs = [int(p[0]) for p in centers_small]
        center_span = float(max(xs) - min(xs)) if len(xs) > 1 else 0.0
        turn_ratio = center_span / max(float(self.input_size[0]), 1.0)
        turn_strength = min(1.0, turn_ratio / max(self.LOOKAHEAD_TURN_TRIGGER_FRAC, 1e-6))
        if (len(left_points_small) > 0) ^ (len(right_points_small) > 0):
            far_frac = self.SINGLE_LANE_TARGET_INDEX_FAR_FRAC
            near_frac = self.SINGLE_LANE_TARGET_INDEX_NEAR_FRAC
        else:
            far_frac = self.TARGET_INDEX_FAR_FRAC
            near_frac = self.TARGET_INDEX_NEAR_FRAC
        frac = far_frac * (1.0 - turn_strength) + near_frac * turn_strength
        return max(0, min(len(centers_small) - 1, int(len(centers_small) * frac)))
    def _offset_target_point(self, target_x: int, target_y: int, crop_h: int):
        return target_x, max(0, target_y - self.TARGET_Y_VISUAL_OFFSET_PX)
    # -----

```

```

# fast control-only API
# -----
def get_steering_error_only(self, frame: np.ndarray, generate_display_data: bool = True):
    """
    Fast path for real-time control.
    Returns only the info needed for steering, avoiding expensive visual generation.
    """
    timings = {}
    t0 = time.perf_counter()
    orig_h, orig_w = frame.shape[:2]
    cut = int(orig_h * self.crop_top_fraction)
    frame_crop = frame[cut:orig_h, :]
    timings["crop_ms"] = (time.perf_counter() - t0) * 1000.0
    t1 = time.perf_counter()
    raw_mask, mask_small = self._predict_mask(frame_crop)
    timings["predict_ms"] = (time.perf_counter() - t1) * 1000.0
    timings["unet_ms"] = timings["predict_ms"]
    t2 = time.perf_counter()
    mask_small = self.postprocess_mask(mask_small)
    timings["postprocess_ms"] = (time.perf_counter() - t2) * 1000.0
    t3 = time.perf_counter()
    if self.prev_frame_left_x is not None and self.prev_frame_right_x is not None:
        prev_center_small = (self.prev_frame_left_x + self.prev_frame_right_x) // 2
    else:
        prev_center_small = None
    centers_small, left_points_small, right_points_small, half_lane_width_small = self.extract_centerline(
        mask_small,
        initial_left_x=self.prev_frame_left_x,
        initial_right_x=self.prev_frame_right_x,
        initial_center_x=prev_center_small,
    )
    timings["centerline_ms"] = (time.perf_counter() - t3) * 1000.0
    crop_h, crop_w = frame_crop.shape[:2]
    img_center_x = crop_w // 2
    target_point = None
    steering_error = None
    centers = []
    left_points = []
    right_points = []
    half_lane_width = None
    mask = None
    if len(centers_small) > 0:
        target_idx = self._target_index(centers_small, left_points_small, right_points_small)
        small_h, small_w = mask_small.shape[:2]
        scale_x = crop_w / float(small_w)
        scale_y = crop_h / float(small_h)
        target_x, target_y = self._scale_point(centers_small[target_idx], scale_x, scale_y)
        target_x += int(round(self._single_lane_target_bias(
            left_points_small,
            right_points_small,
            half_lane_width_small,
            small_w,
        ) * scale_x))
        target_x += int(round(self.target_x_bias_px))
        target_x = max(0, min(crop_w - 1, target_x))

```

```

if self.prev_target_x is None:
    self.prev_target_x, self.prev_target_y = target_x, target_y
else:
    target_x = int(self.TARGET_ALPHA * target_x + (1 - self.TARGET_ALPHA) * self.prev_target_x)
    target_y = int(self.TARGET_ALPHA * target_y + (1 - self.TARGET_ALPHA) * self.prev_target_y)
    self.prev_target_x, self.prev_target_y = target_x, target_y
    target_point = self._offset_target_point(target_x, target_y, crop_h)
    steering_error = target_x - img_center_x
    self.prev_frame_left_x = left_points_small[0][0] if len(left_points_small) > 0 else None
    self.prev_frame_right_x = right_points_small[0][0] if len(right_points_small) > 0 else None
    t4 = time.perf_counter()
    if generate_display_data:
        small_h, small_w = mask_small.shape[:2]
        scale_x = crop_w / float(small_w)
        scale_y = crop_h / float(small_h)
        centers = self._scale_points(centers_small, scale_x, scale_y)
        left_points = self._scale_points(left_points_small, scale_x, scale_y)
        right_points = self._scale_points(right_points_small, scale_x, scale_y)
        half_lane_width = int(round(half_lane_width_small * scale_x))
        mask = cv2.resize(mask_small, (crop_w, crop_h), interpolation=cv2.INTER_NEAREST)
    else:
        half_lane_width = int(round(half_lane_width_small * (crop_w / float(mask_small.shape[1]))))
        timings["scale_and_mask_ms"] = (time.perf_counter() - t4) * 1000.0
        timings["total_ms"] = sum(timings.values())
        single_lane_turn_direction = self._single_lane_turn_direction(left_points_small, right_points_small)
    return {
        "centers": centers,
        "left_points": left_points,
        "right_points": right_points,
        "target_point": target_point,
        "steering_error": steering_error,
        "half_lane_width": half_lane_width,
        "mask": mask,
        "crop_width": crop_w,
        "centers_count_small": len(centers_small),
        "timings": timings,
        "lane_state": {
            "left_count": len(left_points_small),
            "right_count": len(right_points_small),
            "both_visible": bool(len(left_points_small) > 0 and len(right_points_small) > 0),
            "single_visible": bool((len(left_points_small) > 0) ^ (len(right_points_small) > 0)),
            "single_lane_turn_direction": single_lane_turn_direction,
            "inner_lane_missing_turn": bool(single_lane_turn_direction is not None),
        },
    }
# -----
# main API
# -----
def process_frame(self, frame: np.ndarray, draw_debug=True, generate_visuals=True):
    """
    Compatible with your old API, but can skip expensive visuals.
    draw_debug=True:
    draw row points/circles
    generate_visuals=True:
    build overlay/raw_mask_vis/mask visuals for display or saving

```

For fastest Jetson runtime:

process_frame(frame, draw_debug=False, generate_visuals=False)

or use:

get_steering_error_only(frame)

"""

```
orig_h, orig_w = frame.shape[:2]
cut = int(orig_h * self.crop_top_fraction)
frame_crop = frame[cut:orig_h, :]
orig = frame_crop.copy()
raw_mask, mask_small = self._predict_mask(frame_crop)
mask_small = self.postprocess_mask(mask_small)
if self.prev_frame_left_x is not None and self.prev_frame_right_x is not None:
    prev_center_small = (self.prev_frame_left_x + self.prev_frame_right_x) // 2
else:
    prev_center_small = None
centers_small, left_points_small, right_points_small, half_lane_width_small = self.extract_centerline(
    mask_small,
    initial_left_x=self.prev_frame_left_x,
    initial_right_x=self.prev_frame_right_x,
    initial_center_x=prev_center_small,
)
crop_h, crop_w = orig.shape[:2]
small_h, small_w = mask_small.shape[:2]
scale_x = crop_w / float(small_w)
scale_y = crop_h / float(small_h)
centers = self._scale_points(centers_small, scale_x, scale_y)
left_points = self._scale_points(left_points_small, scale_x, scale_y)
right_points = self._scale_points(right_points_small, scale_x, scale_y)
half_lane_width = int(round(half_lane_width_small * scale_x))
target_point = None
steering_error = None
img_center_x = crop_w // 2
if len(centers_small) > 0:
    target_idx = self._target_index(centers_small, left_points_small, right_points_small)
    target_x, target_y = self._scale_point(centers_small[target_idx], scale_x, scale_y)
    target_x += int(round(self._single_lane_target_bias(
        left_points_small,
        right_points_small,
        half_lane_width_small,
        small_w,
    ) * scale_x))
    target_x += int(round(self.target_x_bias_px))
    target_x = max(0, min(crop_w - 1, target_x))
    if self.prev_target_x is None:
        self.prev_target_x, self.prev_target_y = target_x, target_y
    else:
        target_x = int(self.TARGET_ALPHA * target_x + (1 - self.TARGET_ALPHA) * self.prev_target_x)
        target_y = int(self.TARGET_ALPHA * target_y + (1 - self.TARGET_ALPHA) * self.prev_target_y)
        self.prev_target_x, self.prev_target_y = target_x, target_y
    target_point = self._offset_target_point(target_x, target_y, crop_h)
    steering_error = target_x - img_center_x
    self.prev_frame_left_x = left_points_small[0][0] if len(left_points_small) > 0 else None
    self.prev_frame_right_x = right_points_small[0][0] if len(right_points_small) > 0 else None
    overlay = None
raw_mask_vis = None
```

```

vis = None
if generate_visuals:
    mask = cv2.resize(mask_small, (crop_w, crop_h), interpolation=cv2.INTER_NEAREST)
    raw_mask_vis = cv2.resize(raw_mask, (crop_w, crop_h), interpolation=cv2.INTER_LINEAR)
    overlay = orig.copy()
    overlay[mask > 0] = [0, 255, 0]
    vis = overlay.copy()
    vis = self.draw_polyline(vis, left_points, color=(255, 0, 255), thickness=2)
    vis = self.draw_polyline(vis, right_points, color=(0, 165, 255), thickness=2)
    vis = self.draw_polyline(vis, centers, color=(255, 0, 0), thickness=3)
    if draw_debug:
        vis = self.draw_debug_rows(vis, centers, left_points, right_points)
        cv2.line(vis, (img_center_x, 0), (img_center_x, vis.shape[0]), (255, 255, 0), 1)
        if target_point is not None:
            target_x, target_y = target_point
            cv2.circle(vis, (target_x, target_y), 7, (0, 0, 255), -1)
            cv2.putText(
                vis,
                f"Error: {steering_error}px",
                (20, 40),
                cv2.FONT_HERSHEY_SIMPLEX,
                1.0,
                (255, 255, 255),
                2,
                cv2.LINE_AA
            )
    return {
        "cropped_frame": orig,
        "raw_mask_vis": raw_mask_vis,
        "mask": cv2.resize(mask_small, (crop_w, crop_h), interpolation=cv2.INTER_NEAREST),
        "overlay": vis if generate_visuals else None,
        "centers": centers,
        "left_points": left_points,
        "right_points": right_points,
        "target_point": target_point,
        "steering_error": steering_error,
        "half_lane_width": half_lane_width,
    }

```